

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG**

**LÊ HOÀNG BẢO NGỌC
LÊ TRẦN ANH KIỆT
LÊ THANH VY
NGUYỄN VY NGỌC
ĐỖ GIA HUY**

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

**BÁO CÁO ĐỒ ÁN MÔN HỌC
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG**

Thành phố Hồ Chí Minh – Năm 2026

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG**

**BÁO CÁO ĐỒ ÁN MÔN HỌC
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG**

**PHÁT HIỆN VÀ PHÂN LOẠI BIỂN BÁO GIAO THÔNG VIỆT NAM
SỬ DỤNG MÔ HÌNH YOLO**

Họ và tên sinh viên 1: Lê Hoàng Bảo Ngọc

Mã số SV: 22207119

Họ và tên sinh viên 2: Lê Trần Anh Kiệt

Mã số SV: 22207116

Họ và tên sinh viên 3: Lê Thanh Vy

Mã số SV: 22207107

Họ và tên sinh viên 4: Nguyễn Vy Ngọc

Mã số SV: 22207064

Họ và tên sinh viên 5: Đỗ Gia Huy

Mã số SV: 22207038

Thành phố Hồ Chí Minh – Năm 2026

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn đến giảng viên hướng dẫn, người đã định hướng và hỗ trợ chúng em tận tình trong suốt quá trình thực hiện đề tài. Những ý kiến đóng góp của thầy đã giúp chúng em hoàn thiện nội dung nghiên cứu một cách tốt hơn.

Chúng em cũng xin chân thành cảm ơn các thầy cô trong khoa đã trang bị cho chúng em những kiến thức nền tảng cần thiết trong suốt quá trình học tập, tạo điều kiện thuận lợi cho chúng em hiện đồ án.

Do hạn chế về kiến thức và thời gian, báo cáo không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý từ thầy cô để có thể hoàn thiện hơn trong tương lai.

LỜI CAM KẾT

Chúng em xin cam kết rằng toàn bộ nội dung trong báo cáo đồ án này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên. Các nội dung, kết quả và số liệu được trình bày trong báo cáo là trung thực và chưa từng được công bố trong bất kỳ công trình nào trước đây.

Trong quá trình thực hiện đề tài, chúng em đã tham khảo một số tài liệu từ các nguồn khác nhau. Tất cả các tài liệu tham khảo đều đã được trích dẫn rõ ràng và đầy đủ theo quy định. Chúng em hoàn toàn chịu trách nhiệm trước nhà trường về tính trung thực và chính xác của nội dung trong báo cáo này.

Nếu phát hiện có bất kỳ hành vi gian lận nào, chúng em xin hoàn toàn chịu trách nhiệm và chấp nhận mọi hình thức xử lý theo quy định của nhà trường.

BẢNG PHÂN CÔNG NHIỆM VỤ

MSSV	Họ tên	Ngày giao	Công việc	Đánh giá	% Đóng góp
22207119	Lê Hoàng Bảo Ngọc	28/04/2026	Xây dựng và huấn luyện mô hình YOLOv8s	Hoàn thành đúng hạn	20.38%
		29/04/2026	Xây dựng và huấn luyện mô hình YOLOv8m	Hoàn thành đúng hạn	
		01/05/2026	Huấn luyện mô hình YOLOv8s trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		02/05/2026	Huấn luyện mô hình YOLOv8m trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		04/05/2026	Viết word, làm slide phần lựa chọn, xây dựng và huấn luyện mô hình	Hoàn thành đúng hạn	
22207116	Lê Trần Anh Kiệt	28/04/2026	Đánh giá tập dữ liệu và mô hình YOLOv8n	Hoàn thành đúng hạn	20.38%
		29/04/2026	Đánh giá mô hình YOLOv8s. Tìm hiểu phần Augmentation	Hoàn thành đúng hạn	
		30/04/2026	Chuẩn bị xong tập dữ liệu đã Augmentation. Đánh giá tập dữ liệu trước và sau khi Augmentation.	Hoàn thành đúng hạn	
		02/05/2026	Đánh giá kết quả huấn YOLOv8n và YOLOv8s trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		03/05/2026	Đánh giá ma trận nhầm lẫn, hiệu suất chi tiết từng Class qua Bar Chart	Hoàn thành đúng hạn	
		04/05/2026	Viết word, làm slide phần đánh giá Augmentation và đánh giá mô hình	Hoàn thành đúng hạn	
22207107	Lê Thanh Vy	27/04/2026	Lựa chọn mô hình YOLOv8n, xây dựng mô hình	Hoàn thành đúng hạn	20.38%
		28/04/2026	Huấn luyện mô hình YOLOv8n, tìm hiểu thêm các phiên bản YOLOv8 khác	Hoàn thành đúng hạn	
		29/04/2026	Xây dựng và huấn luyện mô hình YOLOv8l	Hoàn thành đúng hạn	

MSSV	Họ tên	Ngày giao	Công việc	Đánh giá	% Đóng góp
		30/04/2026	Huấn luyện mô hình YOLOv8n trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		02/05/2026	Huấn luyện mô hình YOLOv8l trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		04/05/2026	Viết word, làm slide phần lựa chọn, xây dựng và huấn luyện mô hình. Phân công thuyết trình	Hoàn thành đúng hạn	
22207064	Nguyễn Vy Ngọc	29/04/2026	Viết word tổng quan về đề án	Hoàn thành đúng hạn	19.91%
		02/05/2026	Đánh giá kết quả huấn YOLOv8m và YOLOv8l trên tập dữ liệu đã Augmentation	Hoàn thành đúng hạn	
		03/05/2026	So sánh và lựa chọn mô hình tối ưu, đánh giá trade-off	Hoàn thành đúng hạn	
		04/05/2026	Viết word, làm slide phần đánh giá mô hình, đánh giá trade-off	Hoàn thành đúng hạn	
		05/05/2026	Gộp file word của các thành viên, hoàn tất báo cáo	Hoàn thành đúng hạn	
22207038	Đỗ Gia Huy	25/04/2026	Thu thập dữ liệu biển báo giao thông từ các nguồn Kaggle và Roboflow	Hoàn thành đúng hạn	18.96%
		27/04/2026	Gộp hai tập dữ liệu đã tìm và tiền xử lý dữ liệu (remap nhãn, gộp nhãn, cập nhật file class)	Hoàn thành đúng hạn	
		28/04/2026	Phân chia dữ liệu, chuẩn bị xong tập dữ liệu phiên bản 1. Nghiên cứu phần Augmentation (tìm hiểu cách thức Augmentation từ Roboflow)	Hoàn thành đúng hạn phần chuẩn bị dataset lần 1. Nhưng phần Augmentation chưa đạt yêu cầu/tiến độ, bàn giao việc này cho thành viên khác.	
		04/05/2026	Viết word, làm slide phần thu thập và tiền xử lý dữ liệu	Hoàn thành đúng hạn	

TÓM TẮT

Trong bối cảnh bùng nổ về công nghệ trí tuệ nhân tạo và thị giác máy tính, bài toán nhận diện biển báo giao thông đóng vai trò quan trọng trong nhiều ứng dụng thực tiễn (xe tự hành, hệ thống hỗ trợ lái xe nâng cao, giám sát giao thông thông minh,...). Đề tài này tập trung nghiên cứu và xây dựng một hệ thống nhận diện và phân loại biển báo giao thông dựa trên các mô hình học sâu họ YOLOv8. Hệ thống được thiết kế để thực hiện hai nhiệm vụ chính là phát hiện vị trí biển báo trong ảnh và xác định loại biển báo tương ứng.

Trong quá trình thực hiện, đề tài tiến hành huấn luyện và so sánh bốn mô hình YOLOv8 (YOLOv8n, YOLOv8s, YOLOv8m và YOLOv8l) trên cùng một tập dữ liệu nhằm đảm bảo tính khách quan. Các mô hình được đánh giá dựa trên nhiều tiêu chí như mAP, Precision, Recall, IoU, thời gian huấn luyện, thời gian suy luận và tính khả thi của mô hình. Kết quả thực nghiệm cho thấy mỗi mô hình có ưu và nhược điểm riêng, thể hiện rõ sự đánh đổi giữa độ chính xác và tốc độ xử lý.

Thông qua việc phân tích và so sánh, đề tài đưa ra lựa chọn mô hình tối ưu phù hợp với mục tiêu ứng dụng và điều kiện tài nguyên phần cứng. Mặc dù vẫn còn một số hạn chế như quy mô dữ liệu chưa lớn và chưa triển khai thực tế, đề tài đã đạt được mục tiêu đề ra và có thể trở thành cơ sở cho các hướng nghiên cứu trong tương lai.

MỤC LỤC

Contents

LỜI CẢM ƠN	3
LỜI CAM KẾT	4
BẢNG PHÂN CÔNG NHIỆM VỤ	5
TÓM TẮT	1
MỤC LỤC	2
DANH MỤC HÌNH	4
DANH MỤC BẢNG	5
CHƯƠNG 1: GIỚI THIỆU	6
1.1. ĐẶT VẤN ĐỀ	6
1.2. MỤC TIÊU ĐỀ TÀI	6
1.3. PHẠM VI NGHIÊN CỨU	7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	8
2.1. CÁC PHIÊN BẢN YOLOv8	8
2.2. CÁC THÔNG SỐ ĐÁNH GIÁ MÔ HÌNH PHÁT HIỆN ĐỐI TƯỢNG	9
2.2.1. Chỉ Số Giao Thoa Trên Tổng (Intersection over Union - IoU)	9
2.2.2. Precision, Recall và F1-Score	10
2.2.3. Mean Average Precision (mAP)	10
2.2.4. Phân Tích Hàm Mất Mát (Loss Functions) và Đồ Thị Huấn Luyện	11
2.2.5. Confusion Matrix (Ma trận nhầm lẫn)	12
CHƯƠNG 3: XÂY DỰNG MÔ HÌNH	15
3.1. TẬP DỮ LIỆU	15
3.1.1. Nguồn Dữ Liệu	15
3.1.2. Tiền Xử Lý Dữ Liệu – Gộp Và Chuẩn Hóa Nhãn	17
3.1.3. Phân Chia Dữ Liệu	19
3.1.4. Tăng Cường Dữ Liệu (Data Augmentation)	21
3.2. THIẾT LẬP MÔI TRƯỜNG	32
3.2.1. Cấu Hình Mô Hình	32
3.2.2. Thiết Lập Tham Số Huấn Luyện	33
3.2.3. Chuẩn Bị Và Phân Chia Dữ Liệu	33

3.2.4. Tăng Cường Dữ Liệu (Data Augmentation)	34
3.3. HUẤN LUYỆN MÔ HÌNH	34
3.3.1. Mô Hình YOLOv8s	34
3.3.2. Mô Hình YOLOv8n.....	37
3.3.3. Mô Hình YOLOv8m.....	39
3.3.4. Mô Hình YOLOv8l.....	41
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ	45
4.1. ĐÁNH GIÁ TẬP DỮ LIỆU	45
4.1.1. Phân Tích Tập Dữ Liệu Biểu Báo Giao Thông Trước Augmentation.....	45
4.1.2. Phân Tích Tập Dữ Liệu Biểu Báo Giao Thông Sau Augmentation	46
4.2. ĐÁNH GIÁ KẾT QUẢ HUẤN LUYỆN VÀ KIỂM TRA	48
4.2.1. Phân tích độ chính xác và sai số huấn luyện.....	48
4.2.2. Phân Tích Hiệu Suất Chi Tiết Từng Class Qua Bar Chart	51
4.2.3. Đánh Giá Ma Trận Nhầm Lẫn Của Dữ Liệu Kiểm Tra	53
4.2.4. Kết Quả Nhận Diện Trên Tập Test	54
4.3. ĐÁNH GIÁ TRADE-OFF	55
4.3.1. Trade-off Giữa Accuracy và Speed	56
4.3.2. Trade-off Giữa Accuracy và Model Size.....	56
4.3.3. Trade-off Giữa Triển Khai Thực Tế	58
CHƯƠNG 5: KẾT LUẬN	59
TÀI LIỆU THAM KHẢO	60

DANH MỤC HÌNH

Hình 2.1. Các phiên bản YOLOv8.....	8
Hình 2.2. Ma trận Nhầm lẫn	12
Hình 2.3. Hệ thống Thông số Đánh giá Mô hình Object Detection (YOLOv8).....	14
Hình 3.1. Minh họa Augmentation Biến đổi Hình học.....	24
Hình 3.2. Minh họa Augmentation Màu sắc & Ánh sáng	25
Hình 3.3. Minh họa Augmentation Nhiễu & Mờ.....	27
Hình 3.4. Minh họa Augmentation Thời tiết	28
Hình 3.5. Minh họa Augmentation Che khuất.....	28
Hình 3.6. Minh họa Augmentation Pipeline Tổng hợp.....	29
Hình 3.7. Tên ảnh gốc và tên ảnh sau khi Augmentation	30
Hình 4.1. Ảnh gốc và các ảnh Augmentation trong tập dữ liệu thực tế.....	47
Hình 4.2. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8n	50
Hình 4.3. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8s.....	50
Hình 4.4. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8m	50
Hình 4.5. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8l	51
Hình 4.6. Ma trận nhầm lẫn của bốn mô hình YOLO (lần lượt từ trái qua phải, từ trên xuống dưới là YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l).....	53
Hình 4.7. Kết quả nhận diện	55

DANH MỤC BẢNG

Bảng 2.1. Các phiên bản YOLOv8 được sử dụng trong đề tài	8
Bảng 2.2. Tóm tắt các chỉ số hiệu suất (KPIs) của mô hình doanai_yolov8n	11
Bảng 3.1. Thông tin về Bộ dữ liệu 1	15
Bảng 3.2. Thông tin về Bộ dữ liệu 2	16
Bảng 3.3. Kết quả sau khi gộp	18
Bảng 3.4. Kết quả phân chia (trước augmentation)	21
Bảng 3.5. Tổng hợp các Kỹ thuật Tăng cường Dữ liệu được áp dụng	23
Bảng 3.6. Kết quả phân chia (sau augmentation)	31
Bảng 3.7. Tổng kết bước chuẩn bị dữ liệu	32
Bảng 3.8. Tham số huấn luyện	33
Bảng 4.1. Thống kê các lớp đạt $AP@0.5 \geq 95\%$	52
Bảng 4.2. Kết quả đánh giá trên tập test	54
Bảng 4.3. Thống kê các chỉ số so sánh giữa bốn mô hình	57

CHƯƠNG 1: GIỚI THIỆU

1.1. ĐẶT VẤN ĐỀ

Trong bối cảnh bùng nổ nhanh chóng của công nghệ trí tuệ nhân tạo (AI – Artificial Intelligence) và thị giác máy tính (Computer Vision), việc phát triển một mô hình có khả năng nhận diện biển báo giao thông là xu hướng cần thiết. Đặc biệt, trong các hệ thống xe tự hành, khả năng nhận diện và hiểu đúng các biển báo giao thông là yếu tố quan trọng hỗ trợ phương tiện vận hành an toàn. Bên cạnh đó, trong các hệ thống hỗ trợ lái xe nâng cao (ADAS), việc phát hiện sớm các biển báo như giới hạn tốc độ, cấm rẽ hay cảnh báo nguy hiểm giúp người lái có thể phản ứng kịp thời và giảm rủi ro gây tai nạn. Ngoài ra, trong các mô hình giám sát giao thông thông minh, nhận diện biển báo còn góp phần hỗ trợ quản lý và điều phối giao thông hiệu quả.

Tuy nhiên, việc giải quyết bài toán nhận diện biển báo giao thông tại Việt Nam còn nhiều thách thức đặc thù. Hệ thống biển báo đa dạng, gồm 5 nhóm chính như biển cấm, biển nguy hiểm, biển hiệu lệnh, biển chỉ dẫn và biển phụ, mỗi nhóm lại có nhiều thông tin hiển thị khác nhau. Không những vậy, điều kiện thực tế khi thu thập dữ liệu phần lớn không ở dạng lý tưởng (ánh sáng yếu, biển báo bị che khuất một phần, góc chụp nghiêng hoặc bị nhiễu bởi môi trường xung quanh,...). Những yếu tố này là thành phần nhiễu gây ảnh hưởng đến độ chính xác của các mô hình nhận diện nếu không được xử lý phù hợp. Chính vì vậy, việc nghiên cứu và xây dựng một hệ thống nhận diện biển báo giao thông hiệu quả, có khả năng hoạt động tốt trong điều kiện thực tế là một vấn đề mang tính cấp thiết và có ý nghĩa thực tiễn cao.

1.2. MỤC TIÊU ĐỀ TÀI

Mục tiêu chính của đề tài này là xây dựng một hệ thống có khả năng phát hiện và phân loại biển báo giao thông Việt Nam dựa trên mô hình YOLO. Cụ thể, hệ thống cần thực hiện hai nhiệm vụ chính bao gồm phát hiện vị trí của biển báo trong ảnh (Object

Detection) và xác định loại biển báo tương ứng. Việc kết hợp hai nhiệm vụ này giúp hệ thống không chỉ xác định được sự xuất hiện mà còn hiểu được ý nghĩa của biển báo.

Trong quá trình thực hiện, đề tài tập trung so sánh hiệu năng của bốn biến thể thuộc họ mô hình YOLOv8 (YOLOv8n, YOLOv8s, YOLOv8m và YOLOv8l). Đây là những mô hình có sự khác biệt về độ phức tạp và khả năng xử lý dữ liệu, từ đó có thể đánh giá được sự khác biệt về hiệu năng và tài nguyên sử dụng.

Việc đánh giá và lựa chọn mô hình tối ưu nhất dựa trên nhiều tiêu chí khác nhau nhằm đảm bảo tính toàn diện. Các tiêu chí đánh giá gồm độ chính xác trung bình (mAP), Precision và Recall để kiểm tra chất lượng dự đoán, chỉ số IoU để đo lường mức độ trùng khớp giữa dự đoán và thực tế, thời gian huấn luyện và thời gian suy luận để đánh giá hiệu suất và khả năng triển khai trong môi trường thực tế. Thông qua quá trình so sánh này, đề tài hướng đến việc lựa chọn được mô hình phù hợp nhất với yêu cầu của bài toán.

1.3. PHẠM VI NGHIÊN CỨU

Đề tài chỉ sử dụng các mô hình thuộc họ YOLOv8 để thực hiện bài toán nhận diện biển báo giao thông và không mở rộng sang các kiến trúc khác nhằm đảm bảo tính khả thi vì quỹ thời gian nghiên cứu có hạn. Ngoài ra, hệ thống không được triển khai trên các thiết bị nhúng thực tế mà chỉ dừng lại ở mức thử nghiệm và đánh giá trên môi trường máy tính cá nhân.

Bên cạnh đó, những dữ liệu được sử dụng giới hạn trong bộ dữ liệu đã thu thập, gồm dữ liệu tự thu thập và từ các nguồn công khai (không mở rộng quy mô ở mức lớn hơn do hạn chế về thời gian và tài nguyên). Do đó, kết quả của đề tài chủ yếu chỉ mang tính nghiên cứu và tham khảo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. CÁC PHIÊN BẢN YOLOv8

Cơ chế Anchor-free và Hàm mất mát (Loss Function)

YOLOv8 loại bỏ các mỏ neo định sẵn (Anchor-based) để chuyển sang cơ chế Anchor-free. Thay vì dựa trên các khung hình mẫu, mô hình dự đoán trực tiếp tâm của đối tượng. Cải tiến này giúp giảm số lượng khung bao dự đoán dư thừa, đơn giản hóa quy trình hậu xử lý NMS (Non-Maximum Suppression) và tăng tính tổng quát khi đối mặt với các đối tượng có hình dạng biến thiên.

Về tối ưu hóa, YOLOv8 sử dụng các thành phần mất mát phức hợp:

Localization Loss: Sử dụng CIoU (Complete Intersection over Union) để tối ưu hóa độ chính xác của khung bao, tính đến tỷ lệ khung hình và khoảng cách tâm.

Classification Loss: Sử dụng Binary Cross-Entropy để phân loại chính xác các nhãn biến báo.

Mô hình	Kích thước (pixels)	mAP ^{val} 50-95	Tốc độ CPU ONNX (ms)	Tốc độ A100 TensorRT (ms)	Tham số (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Hình 2.1. Các phiên bản YOLOv8

Hiệu suất của YOLOv8 được điều chỉnh thông qua hai tham số tỷ lệ: `depth_multiple` (độ sâu lớp) và `width_multiple` (số lượng kênh đặc trưng).

Bảng 2.1. Các phiên bản YOLOv8 được sử dụng trong đề tài

Phiên bản	Đặc điểm	Ưu điểm
YOLOv8n	Nano	Tốc độ cực cao, FPS lớn trên thiết bị nhúng, Ít tham số
YOLOv8s	Small	Cân bằng tốt giữa tốc độ và độ chính xác
YOLOv8m	Medium	Độ chính xác cao, phù hợp cho Server trung bình
YOLOv8l	Large	Độ chính xác tối ưu, xử lý tốt các trường hợp khó

2.2. CÁC THÔNG SỐ ĐÁNH GIÁ MÔ HÌNH PHÁT HIỆN ĐỐI TƯỢNG

Trong bài toán nhận diện và phân loại biển báo giao thông, việc đánh giá hiệu suất mô hình học sâu đòi hỏi một phương tiếp cận đa chiều nhằm định lượng chính xác năng lực thực thi. Một mô hình Object Detection tối ưu phải giải quyết đồng thời hai nhiệm vụ hệ trọng: xác định vị trí vùng chứa đối tượng (Localization) và phân loại chính xác nhãn thực thể (Classification). Để đo lường sự cân bằng này, các chỉ số kỹ thuật tiêu chuẩn được sử dụng để đánh giá mô hình, đảm bảo tính tin cậy khi triển khai vào các hệ thống hỗ trợ lái xe nâng cao (ADAS) trong điều kiện giao thông phức tạp tại Việt Nam.

2.2.1. Chỉ Số Giao Thoa Trên Tổng (Intersection over Union - IoU)

IoU là thước đo nền tảng nhằm xác định độ chính xác của việc hồi quy vùng chứa (bounding box). Chỉ số này phản ánh mức độ trùng khớp giữa vùng dự đoán của mô hình (Prediction) và vùng thực tế được gán nhãn (Ground Truth).

Công thức toán học của

$$IoU = \frac{Area(Prediction \cap GroundTruth)}{Area(Prediction \cup GroundTruth)}$$

Trong đó:

- $Area(Prediction \cap GroundTruth)$: Diện tích vùng giao giữa dự đoán và thực tế.
- $Area(Prediction \cup GroundTruth)$: Diện tích vùng hợp nhất của hai vùng trên.

Giá trị IoU dao động trong khoảng $[0, 1]$. Trong thực nghiệm phát hiện đối tượng, một ngưỡng (threshold) cố định — thường là 0.5 — được thiết lập để xác định một dự đoán là Dương tính đúng (True Positive – TP). Nếu $\text{IoU} \geq 0.5$, mô hình được công nhận là đã định vị đối tượng thành công.

2.2.2. Precision, Recall và F1-Score

Để đánh giá khả năng phân loại và độ tin cậy trên bộ dữ liệu gồm 12.305 hình ảnh, các chỉ số thống kê sau đây được phân tích kỹ lưỡng:

- **Precision (Độ chính xác):** Tỷ lệ các dự đoán dương tính là chính xác. Chỉ số này phản ánh năng lực của mô hình trong việc hạn chế các trường hợp Dương tính giả (False Positive - FP), giúp giảm thiểu các cảnh báo sai gây nhiễu cho người lái.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Độ nhạy):** Tỷ lệ các đối tượng thực tế được mô hình phát hiện thành công. Trong an toàn giao thông, Recall mang ý nghĩa sống còn vì việc bỏ sót đối tượng (False Negative - FN) có thể dẫn đến rủi ro và chạm nghiêm trọng.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Giá trị trung bình điều hòa giữa Precision và Recall, cung cấp một cái nhìn tổng quát về hiệu năng khi cân cân bằng giữa việc tránh báo động giả và tránh bỏ sót đối tượng.

2.2.3. Mean Average Precision (mAP)

mAP là chỉ số "tiêu chuẩn vàng" (Gold Standard) trong các kiến trúc YOLO, được tính bằng giá trị trung bình của Average Precision (AP) trên tất cả các lớp đối tượng. Dựa trên kết quả huấn luyện mô hình, hai biến thể mAP quan trọng được ghi nhận:

- **mAP@50**: Độ chính xác trung bình tại ngưỡng IoU = 0,5.
- **mAP@50-95**: Giá trị trung bình của mAP tại các ngưỡng IoU từ 0,5 đến 0,95 với bước nhảy 0,05. Đây là chỉ số khắt khe, đánh giá khả năng tinh chỉnh vị trí bounding box ở mức độ tinh vi.

Bảng 2.2. Tóm tắt các chỉ số hiệu suất (KPIs) của mô hình doanai_yolov8n

Chỉ số	Ý nghĩa kỹ thuật
Precision	Khả năng lọc nhiễu và tránh cảnh báo sai
Recall	Khả năng phát hiện toàn diện, tránh bỏ sót biển báo
mAP@50	Hiệu suất phát hiện tổng thể tại ngưỡng cơ bản
mAP@50-95	Độ chính xác định vị tinh vi trên nhiều ngưỡng

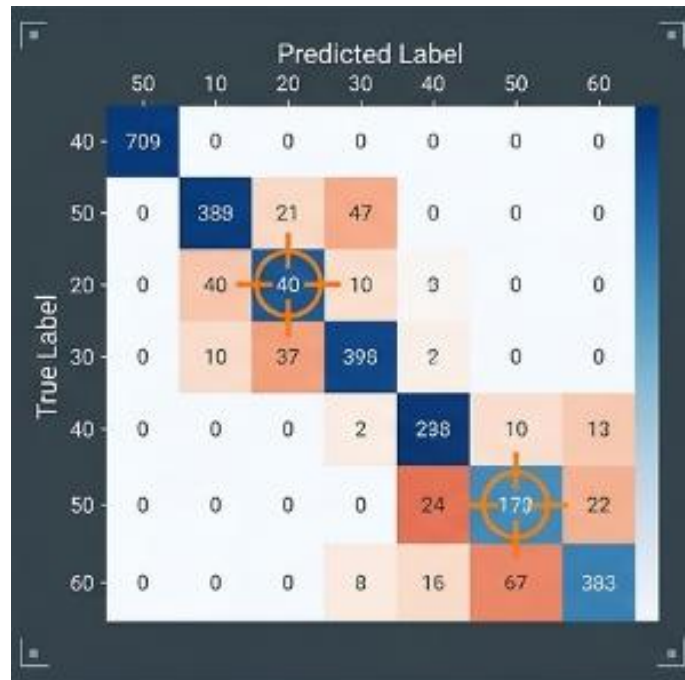
2.2.4. Phân Tích Hàm Mất Mát (Loss Functions) và Đồ Thị Huấn Luyện

Quá trình tối ưu hóa trọng số của mô hình được thực hiện thông qua thuật toán Stochastic Gradient Descent (SGD) với momentum (momentum = 0.937 theo mặc định của YOLOv8), hoặc có thể chuyển sang Adam tùy cấu hình. Hiệu quả của quá trình này được phản ánh qua sự hội tụ của ba thành phần hàm mất mát chính:

- **Box Loss**: Sử dụng để tối ưu hóa sai số hồi quy tọa độ bounding box. Nếu hàm mất mát này giảm đều và duy trì mức thấp, chứng tỏ mô hình đã học được cách định vị chính xác vùng bao biển báo.
- **Classification (Cls) Loss**: Đo lường sai số trong việc phân loại 67 lớp. Mô hình trích xuất đặc trưng qua backbone và neck, sau đó ánh xạ vào vector xác suất qua hàm softmax để phân tách các nhãn có hình thái tương đồng. Sự hội tụ nhanh của Cls Loss sẽ cho thấy mô hình phân loại tốt giữa các nhóm này.
- **Distribution Focal Loss (DFL)**: Đây là thành phần đặc trưng trong kiến trúc anchor-free của YOLOv8. Thay vì dự đoán trực tiếp tọa độ bounding box, DFL học phân phối xác suất rời rạc trên tập giá trị khoảng cách từ điểm neo đến 4 cạnh của bounding box,

từ đó tinh chỉnh vị trí khung bao chính xác hơn, đặc biệt hiệu quả với các biển báo có kích thước nhỏ hoặc bị che khuất một phần.

2.2.5. Confusion Matrix (Ma trận nhầm lẫn)



Hình 2.2. Ma trận Nhầm lẫn

Công cụ trực quan hóa giúp kỹ sư AI xác định các lớp biển báo thường xuyên bị nhận diện sai lẫn nhau (ví dụ: nhầm giữa biển báo giới hạn tốc độ 40km/h và 60km/h do đặc trưng hình học tương đồng).

Ma trận nhầm lẫn (Confusion Matrix) là công cụ trực quan hóa được sử dụng phổ biến trong đánh giá hiệu năng của các mô hình phân loại đa lớp. Về mặt hình thức, ma trận được biểu diễn dưới dạng ma trận vuông kích thước $N \times N$, trong đó N là số lớp cần phân loại, và phần tử C_{ij} thể hiện số lượng (hoặc tỷ lệ) mẫu thuộc lớp thực i được mô hình dự đoán là lớp j .

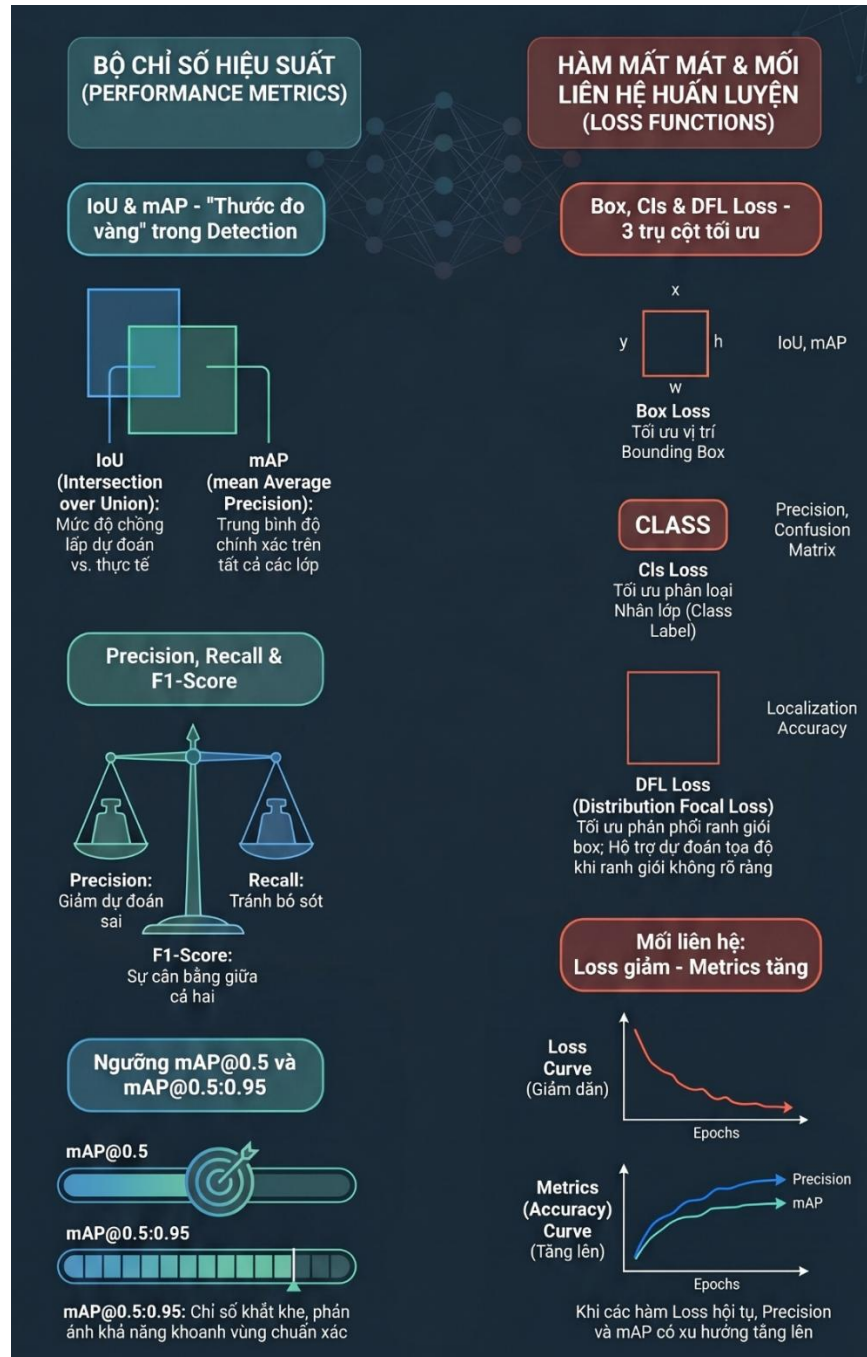
Cấu trúc của ma trận nhầm lẫn bao gồm các thành phần cơ bản sau:

- **True Positive (TP):** Mẫu thuộc lớp i và được dự đoán đúng là lớp i , tương ứng với các phần tử trên đường chéo chính C_{ij} .
- **False Positive (FP):** Mẫu không thuộc lớp i nhưng bị dự đoán nhầm là lớp i , tương ứng với tổng các phần tử theo cột $j=i$ ngoài đường chéo.
- **False Negative (FN):** Mẫu thuộc lớp i nhưng bị dự đoán sang lớp khác, tương ứng với tổng các phần tử theo hàng $i=i$ ngoài đường chéo.

Trong bài toán phát hiện và phân loại đa lớp như nhận diện biển báo giao thông, ma trận nhầm lẫn thường được chuẩn hóa theo hàng (row-normalized) để loại bỏ ảnh hưởng của sự mất cân bằng số lượng mẫu giữa các lớp, theo công thức:

$$\hat{C}_{ij} = \frac{C_{ij}}{\sum_{k=1}^N C_{ik}}$$

Khi đó, giá trị $\hat{C}_{ij}$ phản ánh xác suất có điều kiện $P(\hat{y} = j \mid y = i)$, tức là xác suất mô hình dự đoán lớp j khi nhãn thực sự là lớp i . Đường chéo chính $\hat{C}_{ii}$ càng gần giá trị 1.0 thể hiện khả năng phân loại đúng của mô hình càng cao, trong khi các giá trị lớn ngoài đường chéo chỉ ra sự nhầm lẫn có hệ thống giữa các cặp lớp có đặc trưng hình thái tương đồng.



Hình 2.3. Hệ thống Thông số Đánh giá Mô hình Object Detection (YOLOv8)

CHƯƠNG 3: XÂY DỰNG MÔ HÌNH

3.1. TẬP DỮ LIỆU

3.1.1. Nguồn Dữ Liệu

Bộ dữ liệu sử dụng trong đề tài được tổng hợp từ **hai nguồn độc lập**, sau đó được gộp lại thành một bộ dữ liệu thống nhất với 67 nhãn phân loại (classes). Quá trình gộp được thực hiện bằng Python, đảm bảo sự nhất quán về định dạng nhãn theo chuẩn YOLO.

Bộ dữ liệu 1 – Biển báo giao thông Việt Nam

Bảng 3.1. Thông tin về Bộ dữ liệu 1

Thông tin	Chi tiết
Số ảnh	3.216
Số annotations	8.334
Số classes	52 (index 0–51)
Định dạng nhãn	YOLO (.txt)
Nguồn	Thu thập thực tế tại Việt Nam
Tiêu chuẩn biển báo	QCVN 41:2019/BGTVT
Định danh file	Đánh số thứ tự (0001.jpg – 3216.jpg)

Bộ dữ liệu thứ nhất được thu thập thực tế tại Việt Nam dựa trên ảnh chụp các biển báo giao thông theo tiêu chuẩn QCVN 41:2019/BGTVT. Dữ liệu được gán nhãn thủ công theo định dạng YOLO với 52 loại biển báo, bao gồm:

- Biển cấm (P): P.102, P.103a, P.104, P.106a, P.107, P.111, P.117, P.123a, P.123b, P.124a, P.124b1, P.124c, P.124d, P.127, P.129, P.130, P.131a, P.137, P.139

- Biển nguy hiểm (W): W.201a, W.201b, W.202b, W.203b, W.203c, W.205a, W.205b, W.205c, W.207, W.210, W.221b, W.224, W.225, W.233, W.239b, W.245a, W.246a, W.246c
- Biển hiệu lệnh (R): R.301e, R.302a, R.303, R.415a
- Biển chỉ dẫn (I): I.407a, I.409, I.434a
- Biển phụ (S): S.505a (xe máy, xe tải, xe tải và công)
- Biển đặc biệt: B.8a (cấm xe sơ-mi rơ-moóc), Camera (đường có camera giám sát)

Bộ dữ liệu 2 – Đèn giao thông và Biển tốc độ quốc tế (Roboflow)

Bộ dữ liệu thứ hai được lấy từ nền tảng **Roboflow Universe** – một trong những kho dữ liệu thị giác máy tính công khai uy tín hiện nay.

Bảng 3.2. Thông tin về Bộ dữ liệu 2

Thông tin	Chi tiết
Tên project	Self Driving Cars
Workspace	selfdriving-car-qtywx
Version	6
License	CC BY 4.0
URL	roboflow.com/selfdriving-car-qtywx/self-driving-cars-lfjou
Số ảnh	3.530
Số annotations	4.298
Số classes	15 (index 0–14 gốc, remap thành 52–66)
Định danh file	road*_png.rf.*, *_jpg.rf.* (format Roboflow)

15 class của bộ dữ liệu này bao gồm: *Green Light*, *Red Light*, *Speed Limit* 10/20/30/40/50/60/70/80/90/100/110/120, *Stop*.

3.1.2. Tiền Xử Lý Dữ Liệu – Gộp Và Chuẩn Hóa Nhãn

Vì hai bộ dữ liệu sử dụng hệ thống đánh số class riêng biệt (bộ 1: 0–51, bộ 2: 0–14) nên cần thực hiện bước remap nhãn trước khi gộp để tránh xung đột. Toàn bộ quá trình được thực hiện bằng Python với thư viện chuẩn (os, glob, shutil).

Bước 1 – Remap nhãn bộ dữ liệu mới (0–14 → 52–66)

Mỗi dòng trong file nhãn YOLO có dạng: `<class_id> <cx> <cy> <w> <h>`. Script đọc từng file .txt của bộ mới, cộng thêm 52 vào `class_id`, giữ nguyên tọa độ bounding box, sau đó ghi lại:

```
import glob, os

NEW_LABELS = 'dataset_new/labels'
OUT_LABELS = 'merged/labels'
os.makedirs(OUT_LABELS, exist_ok=True)

for f in glob.glob(f'{NEW_LABELS}/*.txt'):
    lines = open(f).readlines()
    new_lines = []
    for line in lines:
        parts = line.strip().split()
        if parts:
            new_idx = int(parts[0]) + 52 # remap 0-14 → 52-66
            new_lines.append(f"{new_idx} {' '.join(parts[1:])}\n")
    out = f"{OUT_LABELS}/{os.path.basename(f)}"
    open(out, 'w').writelines(new_lines)
```

Bước 2 – Gộp nhãn hai bộ dữ liệu

Nhãn bộ cũ được copy trực tiếp (giữ nguyên index 0 – 51), nhãn bộ mới sau khi remap được copy vào cùng thư mục output:

```
import shutil, glob

OLD_LABELS = 'dataset_old/labels'

# Copy nhãn bộ cũ (giữ nguyên)
for f in glob.glob(f'{OLD_LABELS}/*.txt'):
    shutil.copy(f, f"{OUT_LABELS}/{os.path.basename(f)}")

# Copy ảnh bộ mới
for img in glob.glob('dataset_new/images/*'):
    shutil.copy(img, f"merged/images/{os.path.basename(img)}")
```

Bước 3 – Cập nhật file classes

Sau khi gộp, các file *classes.txt*, *classes_en.txt*, *classes_vie.txt* và *data.yaml* được cập nhật bằng cách nối thêm 15 class mới vào cuối 52 class cũ:

```
new_classes = ['TL.Green', 'TL.Red', 'SL.10', 'SL.100', 'SL.110',
               'SL.120', 'SL.20', 'SL.30', 'SL.40', 'SL.50',
               'SL.60', 'SL.70', 'SL.80', 'SL.90', 'R.Stop']

old_classes = open('classes.txt').read().strip().split('\n')
merged = old_classes + new_classes # 52 + 15 = 67 classes

open('merged/classes.txt', 'w').write('\n'.join(merged))
```

Bảng 3.3. Kết quả sau khi gộp

Thông kê	Bộ cũ	Bộ mới	Sau gộp
Số ảnh	3.216	3.530	6.746
Số file nhãn	3.216	3.530	6.746
Số annotations	8.334	4.298	12.632
Classes (index)	52 (0 – 51)	15 (0 – 14 → 52 – 66)	67 (0 – 66)

3.1.3. Phân Chia Dữ Liệu

Dữ liệu được phân chia theo tỉ lệ 80% – 10% – 10% cho ba tập Train / Validation / Test. Thay vì phân chia ngẫu nhiên đơn thuần, phương pháp Stratified Split được áp dụng để đảm bảo mỗi tập đều có đủ mẫu đại diện cho tất cả 67 classes, đặc biệt quan trọng với các class có số lượng ảnh rất ít (như W.233 chỉ có 3 ảnh, W.246c chỉ có 5 ảnh).

Thuật toán *Stratified Split*

Quá trình phân chia được thực hiện qua các bước:

- Xây dựng bảng ánh xạ: mỗi file ảnh → tập hợp các class xuất hiện trong file đó.
- Shuffle toàn bộ danh sách file với random seed cố định (seed = 42) để đảm bảo tái lập.
- Chia theo tỉ lệ 80-10-10.
- Kiểm tra từng class: nếu class nào vắng mặt trong tập Test hoặc Valid, chuyển 1 file chứa class đó từ Train sang tập thiếu (ưu tiên file ít class nhất để giảm ảnh hưởng đến Train).
- Lặp lại bước kiểm tra cho đến khi tất cả 67 classes có mặt đầy đủ trong cả 3 tập.

```
import glob, os, random
from collections import defaultdict

random.seed(42)
label_dir = 'merged/labels'

# Build map: filename → set of classes
file_classes = {}
class_files = defaultdict(list)
for f in glob.glob(f'{label_dir}/*.txt'):
    fname = os.path.basename(f).replace('.txt', '.jpg')
    cls_set = set()
```

```

    for line in open(f):
        if line.strip(): cls_set.add(int(line.split()[0]))
    file_classes[fname] = cls_set
    for c in cls_set: class_files[c].append(fname)

# Shuffle và chia
all_fnames = list(file_classes.keys())
random.shuffle(all_fnames)
total      = len(all_fnames)
n_train    = int(total * 0.8)
n_test     = int(total * 0.1)
train_set  = set(all_fnames[:n_train])
test_set   = set(all_fnames[n_train:n_train+n_test])
val_set    = set(all_fnames[n_train+n_test:])

# Fix: đảm bảo mỗi class có trong cả 3 tập
for c in sorted(class_files.keys()):
    for target in [test_set, val_set]:
        if not any(f in target for f in class_files[c]):
            candidates = sorted(
                [f for f in class_files[c] if f in train_set],
                key=lambda f: len(file_classes[f]))
            if candidates:
                train_set.remove(candidates[0])
                target.add(candidates[0])

# Ghi kết quả
os.makedirs('split', exist_ok=True)
open('split/train_files.txt', 'w').write('\n'.join(sorted(train_set)))
open('split/valid_files.txt', 'w').write('\n'.join(sorted(val_set)))
open('split/test_files.txt', 'w').write('\n'.join(sorted(test_set)))

```

Bảng 3.4. Kết quả phân chia (trước augmentation)

Tập dữ liệu	Số ảnh	Tỉ lệ	Classes đủ
Train	5.391	79,9%	✓ 67/67
Validation	678	10,1%	✓ 67/67
Test	677	10,0%	✓ 67/67
Tổng	6.746	100%	✓ 67/67

3.1.4. Tăng Cường Dữ Liệu (Data Augmentation)

Trong quá trình xây dựng bộ dữ liệu nhận diện biển báo giao thông Việt Nam, một thách thức lớn được xác định là sự mất cân bằng nghiêm trọng giữa các class: một số class có số lượng ảnh rất lớn (ví dụ class 10 – “No Stopping and No Parking” với 825 ảnh), trong khi nhiều class thiếu số chỉ có từ 1 đến 30 ảnh trong tập gốc (ví dụ class 44 – “Other Danger” chỉ có 1 ảnh, class 21 – “Narrow Road on the Left” chỉ có 8 ảnh). Tình trạng này có thể khiến mô hình YOLOv8 bị thiên lệch, học tốt các class phổ biến nhưng bỏ qua hoặc phân loại sai các class hiếm.

Để khắc phục vấn đề này, một pipeline tăng cường dữ liệu tự động (Data Augmentation Pipeline) được xây dựng bằng Python, sử dụng thư viện *albumentations* kết hợp với *OpenCV*. Pipeline này không chỉ đơn giản nhân bản ảnh mà còn áp dụng hàng loạt biến đổi hình học, màu sắc, nhiễu và điều kiện thời tiết nhằm tạo ra các mẫu ảnh đa dạng, sát với thực tế triển khai trên đường phố Việt Nam.

3.1.4.1. Cấu hình Pipeline và Chiến lược Augment

Pipeline được tổ chức thành một script Python duy nhất gồm nhiều hàm chức năng riêng biệt, chạy tuần tự theo 4 bước chính. Trước tiên, các siêu tham số điều khiển toàn bộ quá trình được định nghĩa tập trung:

```
# — CẤU HÌNH TOÀN BỘ PIPELINE —
NUM_CLASSES          = 67      # tổng số class biển báo VN
MIN_IMAGES_PER_CLASS = 80      # class < ngưỡng này → augment
AUG_MULTIPLIER        = 6      # mỗi ảnh gốc sinh tối đa 6 ảnh aug
TARGET_PER_CLASS      = 150    # mục tiêu tối thiểu mỗi class sau aug
SPLIT_RATIO           = (0.80, 0.10, 0.10) # train : valid : test
MIN_PER_CLASS_SPLIT   = 10     # tối thiểu N ảnh/class trong valid & test
SEED = 42              # đảm bảo tái lập kết quả
```

Chiến lược tăng cường dữ liệu chỉ áp dụng cho các class thiếu số, cụ thể là những class có số ảnh trong tập train nhỏ hơn ngưỡng $MIN_IMAGES_PER_CLASS = 80$ ảnh. Với mỗi class thiếu, số ảnh cần tạo thêm được tính theo công thức:

```
n_need = max(TARGET_PER_CLASS - n_existing, n_existing × AUG_MULTIPLIER)
```

Cách tính này đảm bảo mỗi class đạt tối thiểu 150 ảnh, hoặc nhiều hơn nếu nhân hệ số 6 lên số ảnh gốc cho kết quả lớn hơn.

3.1.4.2. Các Kỹ thuật Tăng cường Dữ liệu được áp dụng

Toàn bộ các phép biến đổi được đóng gói trong hàm `build_transform()`, trả về một đối tượng `A.Compose` kết hợp nhiều kỹ thuật thuộc 5 nhóm chính. Điểm đặc biệt quan trọng là pipeline sử dụng `BboxParams` với `format='yolo'` và `min_visibility=0.35`, đảm bảo nhãn bounding box (định dạng YOLO) luôn được biến đổi đồng bộ với ảnh và loại bỏ các bounding box bị che khuất quá 65% sau khi biến đổi.

Bảng 3.5. Tổng hợp các Kỹ thuật Tăng cường Dữ liệu được áp dụng

Nhóm kỹ thuật	Phương pháp cụ thể	Tham số chính	Mục đích
Hình học	Affine (dịch chuyển, tỉ lệ, xoay)	translate $\pm 5\%$, scale [0.88, 1.12], rotate $\pm 15^\circ$	Mô phỏng camera đặt lệch, biến ở góc độ khác nhau
Hình học	Perspective (phối cảnh)	scale [0.02, 0.05]	Mô phỏng góc nhìn của camera trên xe
Hình học	HorizontalFlip (lật ngang)	p=0.2	Tăng đa dạng góc nhìn trái/phải
Màu sắc & ánh sáng	RandomBrightnessContrast	brightness ± 0.35 , contrast ± 0.3	Mô phỏng điều kiện sáng ban ngày, chiều tối, trời mây
Màu sắc & ánh sáng	HueSaturationValue	hue ± 12 , sat ± 35 , val ± 25	Biến đổi màu sắc do thời tiết và camera khác nhau
Màu sắc & ánh sáng	CLAHE (tăng tương phản cục bộ)	clip_limit=2.5	Làm rõ chi tiết trong ảnh thiếu sáng
Màu sắc & ánh sáng	RGBShift (dịch kênh màu)	± 15 mỗi kênh	Mô phỏng sai lệch white balance của camera
Nhiều & Mờ	GaussNoise / ISONoise / MultiplicativeNoise	OneOf, p=0.35	Mô phỏng nhiễu cảm biến camera ở điều kiện thiếu sáng
Nhiều & Mờ	MotionBlur / GaussianBlur	OneOf, p=0.25	Mô phỏng rung camera, xe chạy tốc độ cao
Thời tiết	RandomRain / RandomFog / RandomSunFlare	OneOf, p=0.2	Mô phỏng điều kiện mưa, sương mù, nắng chói
Che khuất	CoarseDropout (che vùng ngẫu nhiên)	1-4 ô, 8-18px/ô	Mô phỏng vật che khuất một phần biển báo

Nhóm Biến đổi Hình học

Nhóm này bao gồm ba kỹ thuật chính nhằm mô phỏng sự thay đổi góc nhìn, khoảng cách và vị trí của camera đối với biển báo trong thực tế:

- **Affine (p=0.5):** Áp dụng đồng thời dịch chuyển ảnh theo phương ngang và dọc trong phạm vi $\pm 5\%$, thay đổi tỉ lệ kích thước trong khoảng [0.88, 1.12] (thu nhỏ/phóng to tối

đa 12%), và xoay ảnh trong phạm vi $\pm 15^\circ$. Đây là kỹ thuật mô phỏng hiệu quả nhất các tình huống camera bị lệch góc, biển báo nằm ở các khoảng cách khác nhau từ xe.

- **Perspective ($p=0.3$):** Biến đổi phối cảnh với scale trong khoảng $[0.02, 0.05]$, tạo ra hiệu ứng nhìn biển báo từ các góc độ nghiêng khác nhau, mô phỏng camera gắn trên xe ở các độ cao và góc nghiêng khác nhau.
- **HorizontalFlip ($p=0.2$):** Lật ảnh theo chiều ngang với xác suất thấp (20%) vì nhiều biển báo có tính bất đối xứng (biển cấm rẽ trái/phải). Xác suất thấp này được cân nhắc để tránh tạo ra các mẫu không có ý nghĩa thực tế.



Hình 3.1. Minh họa Augmentation Biến đổi Hình học

🚦 Nhóm Biến đổi Màu sắc và Ánh sáng



Hình 3.2. Minh họa Augmentation Màu sắc & Ánh sáng

Đây là nhóm kỹ thuật quan trọng nhất trong điều kiện giao thông Việt Nam vì biến báo thường được chụp trong nhiều điều kiện ánh sáng khác nhau (ban ngày, chiều tối, trời mây, mưa,...):

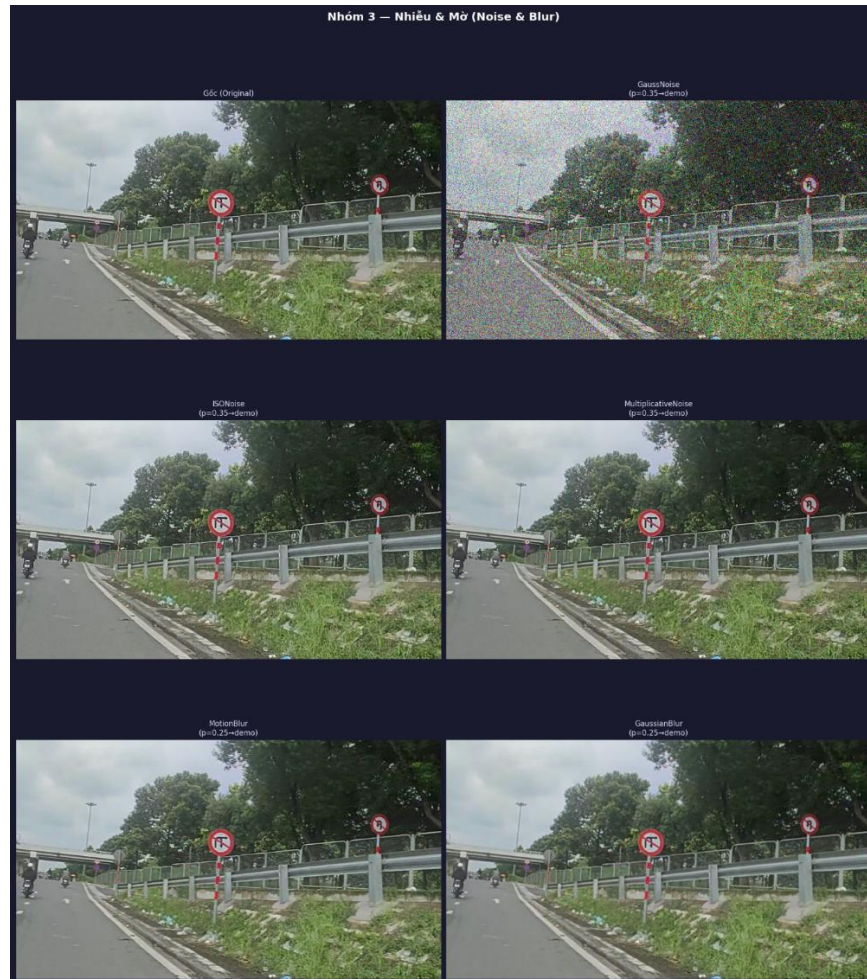
- **RandomBrightnessContrast (p=0.6):** Thay đổi ngẫu nhiên độ sáng ($\pm 35\%$) và độ tương phản ($\pm 30\%$) của ảnh. Xác suất áp dụng cao nhất (60%) trong toàn bộ pipeline vì điều kiện ánh sáng biến đổi nhiều nhất trong thực tế.
- **HueSaturationValue (p=0.5):** Thay đổi sắc độ màu (± 12), độ bão hòa màu (± 35) và giá trị độ sáng (± 25). Kỹ thuật này giúp mô hình không quá phụ thuộc vào màu sắc tuyệt đối của biển báo, vốn có thể thay đổi do điều kiện thời tiết, bụi bẩn, hoặc sai lệch white balance của camera.
- **CLAHE – Contrast Limited Adaptive Histogram Equalization (p=0.3):** Tăng cường tương phản cục bộ với `clip_limit=2.5`, giúp làm rõ chi tiết biển báo trong điều kiện ảnh tối hoặc ngược sáng.
- **RGBShift (p=0.3):** Dịch chuyển độc lập các kênh màu R, G, B trong phạm vi ± 15 , mô phỏng sai lệch white balance của các loại camera khác nhau được lắp đặt trên các phương tiện giao thông.

Nhóm Nhiễu và Mờ ảnh

Hai nhóm OneOf được sử dụng, trong đó mỗi nhóm chọn ngẫu nhiên một kỹ thuật để áp dụng:

- **Nhiễu (p=0.35):** OneOf chọn ngẫu nhiên một trong ba loại nhiễu – GaussNoise (nhiễu Gaussian thuần túy), ISONoise (mô phỏng nhiễu ISO của camera với `color_shift=[0.01, 0.05]` và `intensity=[0.1, 0.4]`) hoặc MultiplicativeNoise (nhiễu nhân, `multiplier=[0.9, 1.1]`). Nhóm này mô phỏng chất lượng cảm biến camera trong điều kiện thiếu sáng hoặc camera chất lượng thấp.
- **Mờ ảnh (p=0.25):** OneOf chọn giữa MotionBlur (mờ do chuyển động, `blur_limit=5`) và GaussianBlur (mờ Gaussian, `blur_limit=3`). MotionBlur mô phỏng tình huống xe

chạy với tốc độ cao hoặc camera bị rung, còn GaussianBlur mô phỏng lens camera không lấy nét hoàn toàn.



Hình 3.3. Minh họa Augmentation Nhiễu & Mờ

🌈 Nhóm Điều kiện Thời tiết

OneOf với xác suất $p=0.2$ chọn ngẫu nhiên một trong ba điều kiện thời tiết – RandomRain với `rain_type="drizzle"` (mưa nhỏ đặc trưng của khí hậu nhiệt đới Việt Nam), RandomFog (sương mù – thường gặp ở các tuyến đường miền núi phía Bắc và vào mùa lạnh), và RandomSunFlare với `flare_roi=(0, 0, 1, 0.5)` và `src_radius=100` (ánh sáng chói từ mặt trời, đặc biệt phổ biến vào buổi sáng và chiều ở miền Nam Việt Nam).



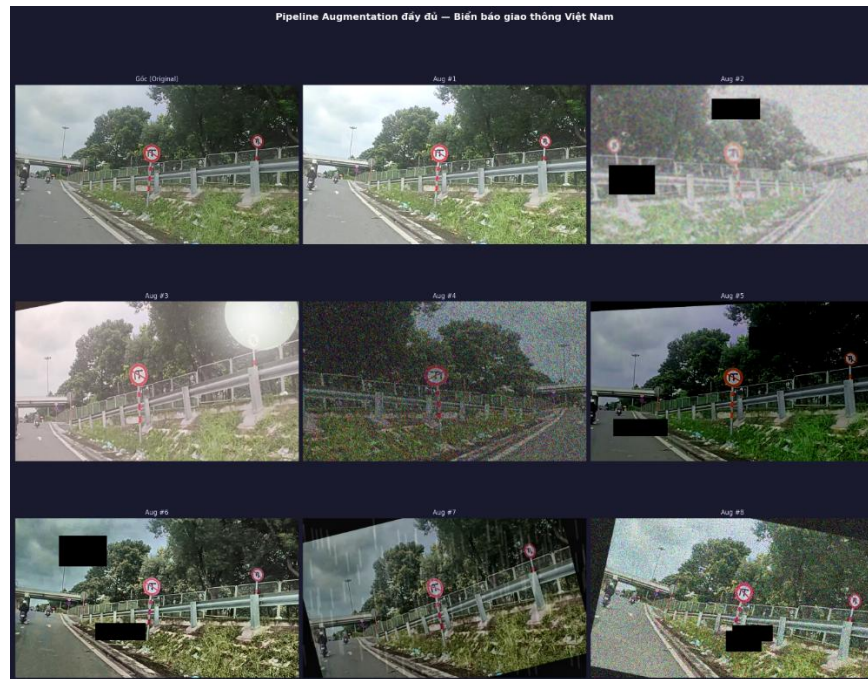
Hình 3.4. Minh họa Augmentation Thời tiết

🌈 Nhóm Che khuất



Hình 3.5. Minh họa Augmentation Che khuất

CoarseDropout ($p=0.2$) tạo ngẫu nhiên từ 1 đến 4 vùng hình chữ nhật màu xám (fill=128) với kích thước từ 8×8 đến 18×18 pixel trên ảnh. Kỹ thuật này mô phỏng các vật cản che một phần biển báo trong thực tế như cành cây, xe khác, hoặc bụi bẩn tích tụ trên biển. Giá trị fill=128 (xám trung tính) được chọn thay vì màu đen hoặc trắng để tránh tạo ra các artifact không tự nhiên.



Hình 3.6. Minh họa Augmentation Pipeline Tổng hợp

3.1.4.3. Các Hàm xử lý Ảnh và Nhãn

Ba hàm tiện ích được xây dựng để đọc, ghi và tìm kiếm file trong quá trình augmentation:

- **read_label(lbl_path):** Đọc file nhãn YOLO (.txt) và trả về danh sách các dòng dưới dạng `[class_id, cx, cy, w, h]`. Hàm này xử lý một trường hợp đặc biệt quan trọng khi `class_id` có thể được lưu dưới dạng `float` (ví dụ: '1.0' thay vì '1') trong một số công cụ gán nhãn và được ép kiểu về `int` để đảm bảo tính nhất quán.

- **write_label(lbl_path, rows):** Ghi danh sách nhãn ra file .txt theo định dạng YOLO chuẩn, với `class_id` là số nguyên và các tọa độ được làm tròn đến 6 chữ số thập phân để đảm bảo độ chính xác.
- **find_image(stem, search_dirs):** Tìm kiếm file ảnh theo tên (không có đuôi file) trong danh sách các thư mục, hỗ trợ nhiều định dạng (.jpg, .jpeg, .png và các biến thể chữ hoa). Hàm này cần thiết vì ảnh gốc và ảnh đã augment nằm ở hai thư mục khác nhau.

Hàm `augment_one(stem, transform, aug_idx, suffix)` thực hiện quá trình augment cho một ảnh đơn lẻ: đọc ảnh bằng OpenCV, chuyển từ BGR sang RGB (do `albumentations` yêu cầu định dạng RGB), đọc nhãn, áp dụng biến đổi đồng bộ cho cả ảnh và bounding box, sau đó lưu ảnh (JPEG chất lượng 92%) và file nhãn với tên mới theo quy tắc `"{stem}_{suffix}_{aug_idx:05d}"`.

```
aug stem: '0104_augc4_00222' → orig_stem: '0104' → in original_stems: True
aug stem: '0104_augc4_00280' → orig_stem: '0104' → in original_stems: True
aug stem: '0104_augc4_00295' → orig_stem: '0104' → in original_stems: True
aug stem: '0104_augc4_00322' → orig_stem: '0104' → in original_stems: True
aug stem: '0104_augc4_00323' → orig_stem: '0104' → in original_stems: True
```

Hình 3.7. Tên ảnh gốc và tên ảnh sau khi Augmentation

3.1.4.4. Quy trình Chạy Pipeline (4 Bước)

Hàm `main()` điều phối toàn bộ quá trình theo 4 bước tuần tự:

- **Bước 1 – Copy và chuẩn hóa dữ liệu gốc (`copy_originals()`):** Sao chép toàn bộ ảnh gốc và file nhãn từ thư mục input sang output. Trong quá trình này, tất cả file nhãn được đọc và ghi lại để chuẩn hóa `class_id` về dạng số nguyên, loại bỏ các dòng lỗi hoặc không hợp lệ.
- **Bước 2 – Augment các class thiểu số (`augment_minority()`):** Xác định các class có số ảnh dưới ngưỡng `MIN_IMAGES_PER_CLASS=80`, tính số ảnh cần tạo thêm cho mỗi class, lặp qua các ảnh thuộc class đó để tạo ảnh augmented. Nếu một class có ít ảnh hơn số

cần tạo, các ảnh gốc được lặp lại nhiều vòng (`pool_cycle`) với thứ tự ngẫu nhiên để đạt đủ số lượng mục tiêu.

- **Bước 3 – Phân chia dữ liệu stratified (`stratified_split()`):** Phân chia toàn bộ dữ liệu (dữ liệu gốc và augmented) thành ba tập train/valid/test theo tỉ lệ 80:10:10. Điểm quan trọng là phương pháp stratified: trước tiên đảm bảo mỗi class có ít nhất `MIN_PER_CLASS_SPLIT=10` ảnh trong cả valid lẫn test, sau đó mới bổ sung thêm ảnh từ phần còn lại để đạt đủ tỉ lệ 10%. Điều này đảm bảo cả 67 class đều xuất hiện trong tập valid và test.
- **Bước 4 – Ghi file split và thống kê (`write_split_files()`, `print_stats()`):** Lưu danh sách tên file (không kèm đuôi) của từng split vào các file *train_files.txt*, *valid_files.txt*, *test_files.txt*. Đồng thời in thống kê cuối cùng bao gồm số ảnh, số class và tổng instances của mỗi tập.

3.1.4.5. Kết quả sau Augmentation và Phân chia lại

Bảng 3.6. Kết quả phân chia (sau augmentation)

Tập dữ liệu	Số ảnh	Số instances	Tỉ lệ	Classes
Train	9.845	26.538	80,0%	✓ 67/67
Validation	1.230	3.959	10,0%	✓ 67/67
Test	1.230	3.936	10,0%	✓ 67/67
Tổng	12.305	34.433	100%	67

So với bộ dữ liệu trước augmentation (6.746 ảnh), tổng số ảnh tăng lên 12.305 ảnh (tăng 82,4%). Đặc biệt, 26 class thiếu dữ liệu đã được bổ sung thêm tổng cộng 5.587 ảnh augmented, giúp cân bằng phân phối dữ liệu giữa các class và cải thiện khả năng tổng quát hóa của mô hình.

Bảng 3.7. Tổng kết bước chuẩn bị dữ liệu

Bước	Nội dung	Kết quả
3.1.1	Thu thập dữ liệu	2 nguồn: biển báo VN (thực tế) + Roboflow (CC BY 4.0)
3.1.2	Tiền xử lý – Gộp dữ liệu	Remap nhãn 0–14 → 52–66, gộp thành 6.746 file, 67 classes
3.1.3	Phân chia dữ liệu	Stratified Split 80-10-10, đảm bảo 67/67 classes mỗi tập
3.1.4	Tăng cường dữ liệu	26 class được augment, +5.587 ảnh, tổng 12.305 ảnh

3.2. THIẾT LẬP MÔI TRƯỜNG

3.2.1. Cấu Hình Mô Hình

Mô hình YOLOv8s (You Only Look Once version 8 - Small): kiến trúc phát hiện đối tượng một giai đoạn (one-stage detector). Phiên bản Small được lựa chọn nhằm cân bằng giữa tốc độ suy luận và độ chính xác, phù hợp với tài nguyên tính toán có hạn.

Mô hình YOLOv8n (You Only Look Once version 8 - Nano): phiên bản nhỏ nhất và nhẹ nhất. So với YOLOv8s, YOLOv8n có số lượng tham số ít hơn đáng kể, tốc độ suy luận nhanh và yêu cầu tài nguyên tính toán thấp nhưng khả năng biểu diễn đặc trưng hạn chế. Đây là lựa chọn phù hợp cho các ứng dụng ưu tiên tốc độ xử lý thời gian thực.

Mô hình YOLOv8m (You Only Look Once version 8 - Medium): phiên bản trung bình, nằm giữa YOLOv8s và YOLOv8l về quy mô tham số. YOLOv8m sở hữu khoảng 25.9 triệu tham số, cho phép mô hình học được các đặc trưng phức tạp hơn từ dữ liệu. Đây là lựa chọn phù hợp khi cần nâng cao độ chính xác mà vẫn đảm bảo tốc độ suy luận ở mức chấp nhận được trên GPU.

Mô hình YOLOv8l (You Only Look Once version 8 - Large): phiên bản lớn thứ hai, đứng trên YOLOv8m và chỉ sau YOLOv8x về quy mô tham số. YOLOv8l sở hữu khoảng 43.7 triệu tham số và 165.7 GFLOPs, cho phép mô hình học được các biểu diễn đặc trưng sâu và phức tạp hơn. Đây là lựa chọn phù hợp khi ưu tiên tối đa hóa độ chính xác trên hệ thống có GPU mạnh, chấp nhận đánh đổi tốc độ suy luận chậm hơn so với các phiên bản nhỏ gọn hơn. Đáng chú ý, với kiến trúc Large, bộ tối ưu hóa được xác định tự động là AdamW (lr=0.000141, momentum=0.9) thay vì SGD mặc định — một điểm khác biệt quan trọng so với các thí nghiệm trước.

3.2.2. Thiết Lập Tham Số Huấn Luyện

Bảng 3.8. Tham số huấn luyện

Tham số	Giá trị	Ghi chú
Số epoch	50	Giới hạn tối đa
Kích thước ảnh đầu vào	640 × 640 px	Chuẩn YOLOv8
Batch size	16 (GPU) / 4 (CPU)	Tự động theo thiết bị
Learning rate	Mặc định Ultralytics (lr0 = 0.01)	Không tinh chỉnh thủ công
Optimizer	SGD (mặc định)	—
Early stopping (patience)	10 epoch	Dừng sớm nếu không cải thiện
Flip ngang (fliplr)	0.0 (tắt)	Phù hợp với biển báo giao thông

Learning rate không được điều chỉnh thủ công mà sử dụng giá trị mặc định của thư viện Ultralytics. Đây là lựa chọn phù hợp trong bước đầu thử nghiệm, tuy nhiên việc tinh chỉnh learning rate có thể giúp cải thiện hiệu quả hội tụ trong các thí nghiệm tiếp theo.

3.2.3. Chuẩn Bị Và Phân Chia Dữ Liệu

Bộ dữ liệu được phân chia theo tỷ lệ 80% – 10% – 10% tương ứng với tập huấn luyện (train), tập kiểm định (validation) và tập kiểm tra (test). Việc phân chia được thực hiện cố định thông qua các file *train_files.txt*, *valid_files.txt* và *test_files.txt* được chuẩn bị trước, đảm bảo tính nhất quán và tránh hiện tượng rò rỉ dữ liệu (data leakage) giữa các tập.

Cấu trúc thư mục dữ liệu tuân theo định dạng chuẩn của Ultralytics YOLO, với nhãn được lưu theo định dạng YOLO (class_id, x_center, y_center, width, height — tọa độ chuẩn hóa).

3.2.4. Tăng Cường Dữ Liệu (Data Augmentation)

Trong quá trình huấn luyện, phép biến đổi lật ngang ảnh (fliplr) được tắt hoàn toàn (fliplr = 0.0). Quyết định này là hợp lý do đặc thù của bài toán nhận dạng biển báo giao thông — việc lật ngang có thể làm sai lệch ngữ nghĩa của một số loại biển báo có tính định hướng (biển chỉ hướng, mũi tên, v.v.). Các kỹ thuật tăng cường dữ liệu còn lại như mosaic, scale, HSV color jitter được giữ ở giá trị mặc định của Ultralytics.

Ngoài ra, bộ dữ liệu gốc có tên DoanAi_aug cho thấy dữ liệu đã được tăng cường từ trước khi đưa vào huấn luyện, do đó việc hạn chế augmentation trong quá trình training là phù hợp để tránh tăng cường dư thừa.

3.3. HUẤN LUYỆN MÔ HÌNH

3.3.1. Mô Hình YOLOv8s

3.3.1.1. Quá trình huấn luyện và Theo dõi

Mô hình được huấn luyện trên GPU của Kaggle (Tesla T4/P100) với cơ chế early stopping được kích hoạt với patience = 10, tức là quá trình huấn luyện sẽ dừng sớm nếu sau 10 epoch liên tiếp không có sự cải thiện về chỉ số validation. Cơ chế này giúp tránh hiện tượng overfitting và tiết kiệm tài nguyên tính toán.

Trong suốt quá trình huấn luyện, hệ thống ghi nhận và lưu trữ các giá trị loss và metric theo từng epoch bao gồm:

- Box Loss (train/box_loss, val/box_loss): đo sai số dự đoán bounding box
- Classification Loss (train/cls_loss, val/cls_loss): đo sai số phân loại nhãn

- DFL Loss (train/df_l_loss, val/df_l_loss): Distribution Focal Loss, hỗ trợ hồi quy bounding box chính xác hơn
- mAP@50, Precision, Recall trên tập validation theo từng epoch

Toàn bộ kết quả được lưu vào file results.csv và được trực quan hóa thành biểu đồ đường (line chart) để theo dõi xu hướng hội tụ của mô hình.

3.3.1.2. Lưu trữ mô hình

Sau mỗi epoch, hệ thống tự động lưu hai checkpoint:

- best.pt: trọng số tốt nhất dựa trên chỉ số validation mAP@50
- last.pt: trọng số của epoch cuối cùng

Mô hình best.pt được sử dụng cho bước đánh giá cuối cùng trên tập test.

3.3.1.3. Nhận xét Training Valid qua từng epoch

🚦 Giai đoạn đầu (Epoch 1–10): Hội tụ nhanh

Đây là giai đoạn mô hình học với tốc độ nhanh nhất. Ngay từ epoch 1, mô hình đã đạt mAP@50 = 0.593 trên tập validation — một điểm khởi đầu khá tốt nhờ trọng số pretrained từ COCO. Chỉ sau 5 epoch, mAP@50 đã vọt lên 0.916, tức tăng hơn 32 điểm phần trăm chỉ trong thời gian ngắn. Song song đó, box loss giảm mạnh từ 1.244 xuống 0.806, cls loss từ 3.6 xuống 0.554, dfl loss từ 1.058 xuống 0.892. Precision và Recall cùng tăng liên tục, phản ánh mô hình đang học được các đặc trưng cơ bản của bộ dữ liệu một cách hiệu quả. Không có dấu hiệu bất thường hay phân kỳ giữa train và val trong giai đoạn này.

🚦 Giai đoạn giữa (Epoch 11–30): Tinh chỉnh và ổn định

Tốc độ cải thiện chậm lại nhưng vẫn duy trì xu hướng tăng ổn định. $mAP@50$ đi từ 0.954 (epoch 11) lên 0.976 (epoch 29), tức chỉ tăng thêm khoảng 2 điểm trong 19 epoch — cho thấy mô hình đang bước vào vùng tối ưu cục bộ. Các giá trị loss tiếp tục giảm đều: box loss về 0.663, cls loss về 0.399, dfl loss về 0.852. Precision đạt ngưỡng ổn định trong vùng 0.95–0.97, Recall dần tiến đến 0.96. Đáng chú ý là val loss không có dấu hiệu tăng trở lại trong giai đoạn này, cho thấy mô hình chưa bị overfitting.

Giai đoạn cuối (Epoch 31–50): Hội tụ và bão hòa

Từ epoch 40, tốc độ cải thiện của $mAP@50$ gần như bão hòa, dao động quanh ngưỡng 0.978–0.980. Tuy nhiên, $mAP@50-95$ vẫn tiếp tục tăng nhẹ và đều đặn, từ 0.776 (epoch 31) lên 0.790 (epoch 50), cho thấy mô hình vẫn đang cải thiện khả năng định vị bounding box chính xác hơn ở các ngưỡng IoU cao. Đáng chú ý là từ epoch 41, augmentation mosaic bị tắt (Closing dataloader mosaic) theo lịch mặc định của Ultralytics — đây là cơ chế giúp mô hình fine-tune trên ảnh gốc không bị biến đổi trong những epoch cuối. Điều này giải thích sự giảm đột ngột của cls loss từ 0.358 (epoch 40) xuống 0.313 (epoch 41). Mô hình kết thúc ở epoch 50 với $mAP@50 = 0.980$ và $mAP@50-95 = 0.790$, Precision = 0.964, Recall = 0.967.

Nhận xét tổng thể về overfitting/underfitting

Trong suốt 50 epoch, không quan sát thấy hiện tượng overfitting đáng kể. Loss trên tập validation giảm song song và nhất quán với loss trên tập train, khoảng cách giữa hai đường loss được duy trì ở mức nhỏ và ổn định. Mô hình không kích hoạt early stopping, cho thấy vẫn còn dư địa cải thiện nếu tăng thêm số epoch. Đây là dấu hiệu tích cực về khả năng tổng quát hóa của mô hình YOLOv8s trên bộ dữ liệu biển báo giao thông này.

3.3.1.4. Đánh giá mô hình

Quá trình đánh giá được thực hiện độc lập trên cả tập validation và tập test sau khi huấn luyện kết thúc, sử dụng checkpoint best.pt. Việc đánh giá tách biệt trên hai tập này

đảm bảo kết quả trên tập test phản ánh khả năng tổng quát hóa thực sự của mô hình trên dữ liệu chưa từng được quan sát trong quá trình huấn luyện. Các chỉ số được báo cáo bao gồm mAP@50, mAP@50-95, Precision và Recall.

3.3.2. Mô Hình YOLOv8n

3.3.2.1. Quá trình huấn luyện và Theo dõi

Mô hình được huấn luyện trên GPU của Kaggle với cơ chế early stopping kích hoạt ở patience = 10. Do YOLOv8n có kiến trúc nhỏ gọn hơn YOLOv8s, thời gian huấn luyện mỗi epoch ngắn hơn đáng kể, tuy nhiên khả năng học các đặc trưng phức tạp cũng bị giới hạn hơn. Điều này có thể ảnh hưởng đến tốc độ hội tụ và ngưỡng hiệu suất tối đa mà mô hình đạt được.

Trong suốt quá trình huấn luyện, hệ thống ghi nhận và lưu trữ các giá trị loss và metric theo từng epoch bao gồm:

- Box Loss (train/box_loss, val/box_loss): đo sai số dự đoán bounding box
- Classification Loss (train/cls_loss, val/cls_loss): đo sai số phân loại nhãn
- DFL Loss (train/dfl_loss, val/dfl_loss): Distribution Focal Loss, hỗ trợ hồi quy bounding box chính xác hơn
- mAP@50, Precision, Recall trên tập validation theo từng epoch

Toàn bộ kết quả được lưu vào file results.csv và được trực quan hóa thành hai nhóm biểu đồ: nhóm loss (Box/Cls/DFL Loss theo epoch) và nhóm metric (mAP@50, Precision, Recall theo epoch), giúp theo dõi toàn diện quá trình hội tụ của mô hình.

3.3.2.2. Lưu trữ mô hình

Sau mỗi epoch, hệ thống tự động lưu hai checkpoint:

- best.pt: trọng số tốt nhất dựa trên chỉ số validation mAP@50
- last.pt: trọng số của epoch cuối cùng

Checkpoint best.pt được sử dụng cho bước đánh giá cuối cùng trên tập test, đảm bảo kết quả báo cáo phản ánh hiệu suất đỉnh của mô hình chứ không phải trạng thái cuối quá trình huấn luyện.

3.3.2.3. Nhận xét Training Valid qua từng epoch

Giai đoạn đầu (Epoch 1–10): Khởi động chậm hơn so với YOLOv8s

Khác với YOLOv8s vốn khởi đầu khá mạnh, YOLOv8n cho thấy điểm xuất phát thấp hơn rõ rệt. Epoch 1 chỉ đạt mAP@50 = 0.176 — thấp hơn gần 42 điểm phần trăm so với YOLOv8s ở cùng epoch (0.593). Điều này phản ánh hạn chế về năng lực biểu diễn đặc trưng của kiến trúc Nano khi tiếp xúc với bộ dữ liệu đa lớp phức tạp. Tuy nhiên, mô hình hội tụ nhanh trong các epoch tiếp theo: đến epoch 5, mAP@50 đạt 0.799 và đến epoch 10 đạt 0.890. Các thành phần loss cũng giảm mạnh trong giai đoạn này, với box loss từ 1.351 xuống 0.918, cls loss từ 4.535 xuống 0.946, dfl loss từ 1.071 xuống 0.907.

Giai đoạn giữa (Epoch 11–30): Tăng trưởng ổn định nhưng chậm dần

Từ epoch 11 đến epoch 30, mAP@50 tăng từ 0.901 lên 0.947 — mức tăng khoảng 4.6 điểm trong 20 epoch. Tốc độ cải thiện chậm hơn đáng kể so với giai đoạn đầu, cho thấy mô hình đang tiến gần đến ngưỡng năng lực tối đa của kiến trúc Nano. Precision dao động quanh vùng 0.92–0.95, Recall tăng dần từ 0.826 lên 0.901. Loss trên cả train và val tiếp tục giảm đều, không có dấu hiệu phân kỳ đáng lo ngại.

Giai đoạn cuối (Epoch 31–50): Bảo hòa rõ rệt

Từ epoch 41, augmentation mosaic bị tắt theo lịch mặc định của Ultralytics, kéo theo sự giảm đột ngột của cls loss từ 0.537 (epoch 40) xuống 0.489 (epoch 41). Tuy nhiên,

mAP@50 trong giai đoạn này chỉ dao động trong khoảng hẹp từ 0.954 đến 0.960, cho thấy mô hình đã đạt trạng thái bão hòa. mAP@50-95 tiếp tục tăng nhẹ từ 0.738 lên 0.745, phản ánh sự cải thiện nhỏ về độ chính xác định vị bounding box. Mô hình kết thúc ở epoch 50 với mAP@50 = 0.960 và mAP@50-95 = 0.745, Precision = 0.947, Recall = 0.921.

3.3.2.4. Đánh giá mô hình

Nhìn chung, YOLOv8n đạt được mức hiệu suất tốt so với quy mô tham số của mô hình. Với tốc độ suy luận chỉ 1.6ms/ảnh trên GPU Tesla T4 và kích thước model chỉ 6.3MB, đây là lựa chọn phù hợp cho các ứng dụng yêu cầu triển khai trên thiết bị nhúng hoặc edge device với tài nguyên hạn chế. Tuy nhiên, để cải thiện hiệu suất trên các lớp khó như đèn tín hiệu giao thông và biển báo tốc độ, cần cân nhắc sử dụng kiến trúc lớn hơn hoặc bổ sung thêm dữ liệu huấn luyện cho các lớp có số lượng mẫu thấp.

3.3.3. Mô Hình YOLOv8m

3.3.3.1. Quá trình huấn luyện và Theo dõi

Mô hình được huấn luyện trên GPU của Kaggle với cơ chế early stopping kích hoạt ở patience = 10, tức là quá trình huấn luyện sẽ dừng sớm nếu sau 10 epoch liên tiếp không có sự cải thiện về chỉ số validation mAP@50. Trong suốt quá trình huấn luyện, hệ thống ghi nhận và lưu trữ các giá trị loss và metric theo từng epoch bao gồm:

- Box Loss (train/box_loss, val/box_loss): đo sai số dự đoán bounding box
- Classification Loss (train/cls_loss, val/cls_loss): đo sai số phân loại nhãn
- DFL Loss (train/dfl_loss, val/dfl_loss): Distribution Focal Loss, hỗ trợ hồi quy bounding box chính xác hơn
- mAP@50, Precision, Recall trên tập validation theo từng epoch

Toàn bộ kết quả được lưu vào file results.csv và được trực quan hóa thành hai nhóm biểu đồ: nhóm loss (Box/Cls/DFL Loss theo epoch) và nhóm metric (mAP@50, Precision, Recall theo epoch).

3.3.3.2. Lưu trữ mô hình

(Phần này sẽ được bổ sung sau khi có dữ liệu từ file results.csv)

3.3.3.3. Nhận xét Training Valid qua từng epoch

Giai đoạn đầu (Epoch 1–10): Hội tụ nhanh và mạnh

Khác biệt rõ rệt nhất so với YOLOv8n là điểm khởi đầu của YOLOv8m cao hơn đáng kể. Ngay từ epoch 1, mô hình đã đạt $\text{mAP}@50 = 0.730$, cao hơn nhiều so với mức 0.176 của YOLOv8n ở cùng epoch, cho thấy kiến trúc Medium với nhiều tham số hơn tận dụng trọng số pretrained từ COCO hiệu quả hơn. Đến epoch 5, $\text{mAP}@50$ đạt 0.933 và đến epoch 10 đạt 0.963. Box loss giảm mạnh từ 1.127 xuống 0.759, cls loss từ 2.648 xuống 0.490, dfl loss từ 1.019 xuống 0.887. Precision và Recall cùng tăng nhanh, lần lượt đạt 0.942 và 0.937 sau 10 epoch.

Giai đoạn giữa (Epoch 11–30): Tinh chỉnh ổn định, cải thiện đều đặn

Từ epoch 11 đến epoch 30, $\text{mAP}@50$ tăng từ 0.964 lên 0.982 — mức tăng khoảng 1.8 điểm trong 20 epoch, phản ánh quá trình tinh chỉnh tham số ở giai đoạn mô hình đã học được các đặc trưng chính. $\text{mAP}@50-95$ cải thiện rõ hơn, từ 0.746 lên 0.797, cho thấy mô hình đang học cách định vị bounding box chính xác hơn ở các ngưỡng IoU cao. Precision ổn định trong vùng 0.956–0.970, Recall tiếp tục tăng đều từ 0.927 lên 0.973. Đáng chú ý, trong giai đoạn này val loss không có dấu hiệu tăng trở lại, xác nhận mô hình chưa bị overfitting.

Giai đoạn cuối (Epoch 31–50): Hội tụ và bão hòa

Từ epoch 41, augmentation mosaic bị tắt theo lịch mặc định của Ultralytics, kéo theo sự giảm đột ngột của cls loss từ 0.303 (epoch 40) xuống 0.257 (epoch 41). Trong giai đoạn này, mAP@50 dao động ổn định quanh ngưỡng 0.983–0.984, trong khi mAP@50-95 vẫn tiếp tục tăng nhẹ từ 0.807 lên 0.812. Precision đạt ngưỡng cao nhất trong toàn bộ quá trình với 0.975 ở epoch 47–48, Recall duy trì ổn định quanh 0.973–0.975. Mô hình kết thúc ở epoch 50 với mAP@50 = 0.983 và mAP@50-95 = 0.812, Precision = 0.974, Recall = 0.973.

3.3.3.4. Đánh giá mô hình

Trong suốt 50 epoch, không quan sát thấy hiện tượng overfitting. Val loss giảm song song với train loss và không có dấu hiệu phân kỳ ở giai đoạn cuối. Với kích thước model 52.1MB và tốc độ suy luận 7.3ms/ảnh trên GPU Tesla T4 — chậm hơn khoảng 4.5 lần so với YOLOv8n (1.6ms/ảnh) — YOLOv8m phù hợp cho các ứng dụng triển khai trên server hoặc hệ thống có GPU, nơi độ chính xác được ưu tiên hơn tốc độ.

3.3.4. Mô Hình YOLOv8l

3.3.4.1. Quá trình huấn luyện và Theo dõi

Mô hình được huấn luyện trên GPU Tesla T4 của Kaggle (14.913 MiB VRAM) với cơ chế early stopping kích hoạt ở patience = 10. Do YOLOv8l có kiến trúc lớn với 43.7 triệu tham số và 165.7 GFLOPs, thời gian huấn luyện mỗi epoch dài hơn đáng kể so với các phiên bản nhỏ hơn (khoảng 8 phút 50 giây đến 8 phút 56 giây mỗi epoch). Trong suốt quá trình huấn luyện, hệ thống ghi nhận và lưu trữ các giá trị loss và metric theo từng epoch bao gồm:

- Box Loss (train/box_loss, val/box_loss): đo sai số dự đoán bounding box
- Classification Loss (train/cls_loss, val/cls_loss): đo sai số phân loại nhãn

- DFL Loss (train/df_l_loss, val/df_l_loss): Distribution Focal Loss, hỗ trợ hồi quy bounding box chính xác hơn
- mAP@50, Precision, Recall trên tập validation theo từng epoch

Toàn bộ kết quả được lưu vào file results.csv và được trực quan hóa thành hai nhóm biểu đồ: nhóm loss (Box/Cls/DFL Loss theo epoch) và nhóm metric (mAP@50, Precision, Recall theo epoch), giúp theo dõi toàn diện quá trình hội tụ của mô hình.

3.3.4.2. Lưu trữ mô hình

Sau mỗi epoch, hệ thống tự động lưu hai checkpoint:

- best.pt: trọng số tốt nhất dựa trên chỉ số validation mAP@50
- last.pt: trọng số của epoch cuối cùng

Checkpoint best.pt được lưu tại /kaggle/working/doanai_yolov8l/weights/best.pt và được sử dụng cho bước đánh giá cuối cùng trên tập test, đảm bảo kết quả báo cáo phản ánh hiệu suất đỉnh của mô hình.

3.3.4.3. Nhận xét Training Valid qua từng epoch

🚦 Giai đoạn đầu (Epoch 1–10): Khởi đầu mạnh mẽ, hội tụ ấn tượng

Với kiến trúc Large và bộ tối ưu hóa AdamW, YOLOv8l thể hiện khả năng khởi đầu vượt trội so với tất cả các phiên bản nhỏ hơn. Ngay từ epoch 1, mô hình đạt mAP@50 = 0.780 — cao hơn đáng kể so với YOLOv8s (0.593), YOLOv8m (0.730) và YOLOv8n (0.176) ở cùng epoch — phản ánh năng lực biểu diễn đặc trưng vượt trội của kiến trúc lớn hơn khi khai thác trọng số pretrained từ COCO. Trong giai đoạn này, các thành phần loss giảm nhanh và mạnh: box loss từ 1.060 (epoch 1) xuống 0.758 (epoch 10), cls loss từ 2.360 xuống 0.485, dfl loss từ 0.998 xuống 0.889. Đến epoch 5, mAP@50 đã đạt 0.937, và đến epoch 10 đạt 0.966. Precision và Recall cùng tăng ổn định, lần lượt đạt 0.948 và 0.935 sau

10 epoch. Đây là giai đoạn hội tụ nhanh nhất và ấn tượng nhất trong chuỗi thí nghiệm so sánh các biến thể YOLOv8.

Giai đoạn giữa (Epoch 11–30): Tình hình bền vững, cải thiện đều đặn ở mAP@50-95

Từ epoch 11 đến epoch 30, mAP@50 tăng từ 0.966 lên 0.981 — mức tăng khoảng 1.5 điểm trong 20 epoch, cho thấy mô hình đã bước vào vùng tinh chỉnh. Tuy nhiên, mAP@50-95 cải thiện rõ rệt hơn, từ 0.756 (epoch 11) lên 0.803 (epoch 30), phản ánh mô hình đang học cách định vị bounding box chính xác hơn ở các ngưỡng IoU cao — điều này nhất quán với hành vi của kiến trúc lớn hơn vốn có khả năng biểu diễn chi tiết tốt hơn. Precision ổn định trong vùng 0.960–0.976, Recall tiếp tục tăng đều từ 0.934 lên 0.965. Các thành phần loss tiếp tục giảm đều: box loss từ 0.735 xuống 0.566, cls loss từ 0.468 xuống 0.331. Đáng chú ý, không quan sát thấy dấu hiệu phân kỳ giữa train loss và val loss trong giai đoạn này, xác nhận mô hình đang tổng quát hóa tốt.

Giai đoạn cuối (Epoch 31–50): Hội tụ chắc chắn, mAP@50-95 vẫn tăng trưởng tích cực

Từ epoch 41, augmentation mosaic bị tắt theo lịch mặc định của Ultralytics (close_mosaic=10), kéo theo sự giảm đột ngột của cls loss từ 0.288 (epoch 40) xuống 0.240 (epoch 41) — hành vi nhất quán với các thí nghiệm trước. mAP@50 trong giai đoạn 31–50 dao động ổn định và tăng dần từ 0.981 lên 0.985, trong khi mAP@50-95 vẫn tiếp tục cải thiện từ 0.804 lên 0.821. Precision duy trì ổn định trong vùng 0.970–0.978, Recall tăng dần đạt mức 0.972–0.978 ở các epoch cuối. Mô hình kết thúc ở epoch 50 với mAP@50 = 0.985 và mAP@50-95 = 0.821, Precision = 0.977, Recall = 0.972.

Nhận xét tổng thể về overfitting/underfitting

Trong suốt 50 epoch, không quan sát thấy hiện tượng overfitting đáng kể. Loss trên tập validation giảm song song và nhất quán với loss trên tập train, khoảng cách giữa hai

đường loss được duy trì ổn định. Mô hình không kích hoạt early stopping, cho thấy vẫn còn dư địa cải thiện nếu tiếp tục huấn luyện với số epoch lớn hơn. Điều này cũng phản ánh khả năng tổng quát hóa tốt của YOLOv8l trên bộ dữ liệu biển báo giao thông đa lớp này — kiến trúc lớn hơn nhưng không bị overfit nhờ bộ dữ liệu đã được tăng cường đủ phong phú và chiến lược regularization hợp lý ($\text{weight_decay}=0.0005$, $\text{dropout}=0.0$, $\text{erasing}=0.4$).

3.3.4.4. Đánh giá mô hình

Quá trình đánh giá được thực hiện độc lập trên cả tập validation và tập test sau khi huấn luyện kết thúc, sử dụng checkpoint best.pt. Trên tập validation (1.230 ảnh, 3.959 instances): $\text{mAP}@50 = 98.5\%$, $\text{mAP}@50-95 = 82.4\%$, Precision = 97.7%, Recall = 97.3%. Trên tập test (1.230 ảnh, 3.936 instances): $\text{mAP}@50 = 98.2\%$, $\text{mAP}@50-95 = 82.3\%$, Precision = 97.6%, Recall = 97.1%. Khoảng cách nhỏ giữa kết quả val và test (chênh lệch chỉ 0.3 điểm $\text{mAP}@50$) xác nhận khả năng tổng quát hóa tốt của mô hình. Tốc độ suy luận đạt 30.0ms/ảnh (inference) trên GPU Tesla T4 — chậm hơn so với YOLOv8m (29.2ms) nhưng đổi lại đạt mức $\text{mAP}@50-95$ cao hơn trên tập test. Với kích thước model 83.7MB và độ chính xác vượt trội, YOLOv8l là lựa chọn phù hợp cho các hệ thống yêu cầu hiệu suất cao nhất, có GPU đủ mạnh, và chấp nhận chi phí tính toán lớn hơn so với các biến thể nhỏ hơn.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. ĐÁNH GIÁ TẬP DỮ LIỆU

4.1.1. Phân Tích Tập Dữ Liệu Biển Báo Giao Thông Trước Augmentation

Bộ dữ liệu biển báo giao thông Việt Nam trước khi tăng cường bao gồm 5.391 ảnh trong tập train, 678 ảnh trong tập valid và 677 ảnh trong tập test, với tổng cộng 67 class biển báo. Tuy nhiên, phân tích phân phối dữ liệu trên tập train cho thấy sự mất cân bằng nghiêm trọng giữa các class, đây là thách thức chính cần giải quyết trước khi đưa vào huấn luyện mô hình YOLOv8.

Sự chênh lệch giữa các nhóm class phổ biến và thiếu số gây ra rủi ro lớn về bias (thiên lệch):

- Nhóm ưu thế: Class 10 ("No Stopping and No Parking") dẫn đầu với 825 ảnh và 859 instances ở tập train.
- Nhóm thiếu số: Class 44 ("Other Danger") chỉ có duy nhất 1 ảnh/1 instance ở tập train. Các class khác như 21 ("Narrow Road on the Left") có 8 ảnh, hay class 51 ("Warning: Obstacle ahead") chỉ có 2 ảnh.

Ở tập dữ liệu gốc quy trình phân chia dữ liệu đã được can thiệp bằng mã nguồn Python để đảm bảo mọi class (kể cả các class cực hiếm như class 44 hay 51) đều xuất hiện ít nhất một lần trong cả ba tập Train, Valid và Test. Mặc dù kết quả huấn luyện trên tập dữ liệu gốc có thể cho ra các chỉ số mAP, Precision và Recall rất cao (mAP@0.5 đạt 98.06% ở YOLOv8 Medium), nhưng việc áp dụng mô hình này vào thực tế là không khả thi vì những lý do sau:

- **Thiếu tính đại diện:** Một biển báo trong thực tế có thể xuất hiện dưới hàng ngàn biến thể về góc nhìn, độ cao, cường độ ánh sáng hay điều kiện thời tiết. Việc chỉ có 1 ảnh

duy nhất trong tập Test để đại diện cho một loại biển báo khiến hệ thống đánh giá không thể khẳng định mô hình có khả năng nhận diện bên phải hay không. Kết quả đạt được trên tập Test lúc này chỉ mang tính chất "may rủi" dựa trên đặc điểm của một tấm ảnh cụ thể đó.

- **Chỉ số đánh giá bị cực đoan hóa:** Với các class chỉ có 1 mẫu thử, các chỉ số như Precision hay Recall chỉ có hai giá trị cực đoan: 100% (nếu nhận diện đúng mẫu đó) hoặc 0% (nếu nhận diện sai). Điều này làm nhiễu loạn biểu đồ mAP tổng thể và không phản ánh đúng năng lực học thực sự của mô hình YOLOv8 đối với các biển báo thiếu số.
- **Rủi ro mất an toàn trong triển khai:** Nếu mẫu duy nhất trong tập Test là một ảnh ở điều kiện lý tưởng (rõ nét, đủ sáng), mô hình có thể đạt điểm tuyệt đối trong báo cáo. Tuy nhiên, khi xe tự hành gặp đúng biển báo đó nhưng bị che khuất một phần hoặc trong đêm tối, mô hình có nguy cơ thất bại hoàn toàn vì chưa bao giờ được kiểm thử qua các biến thể khác nghiệt hơn.

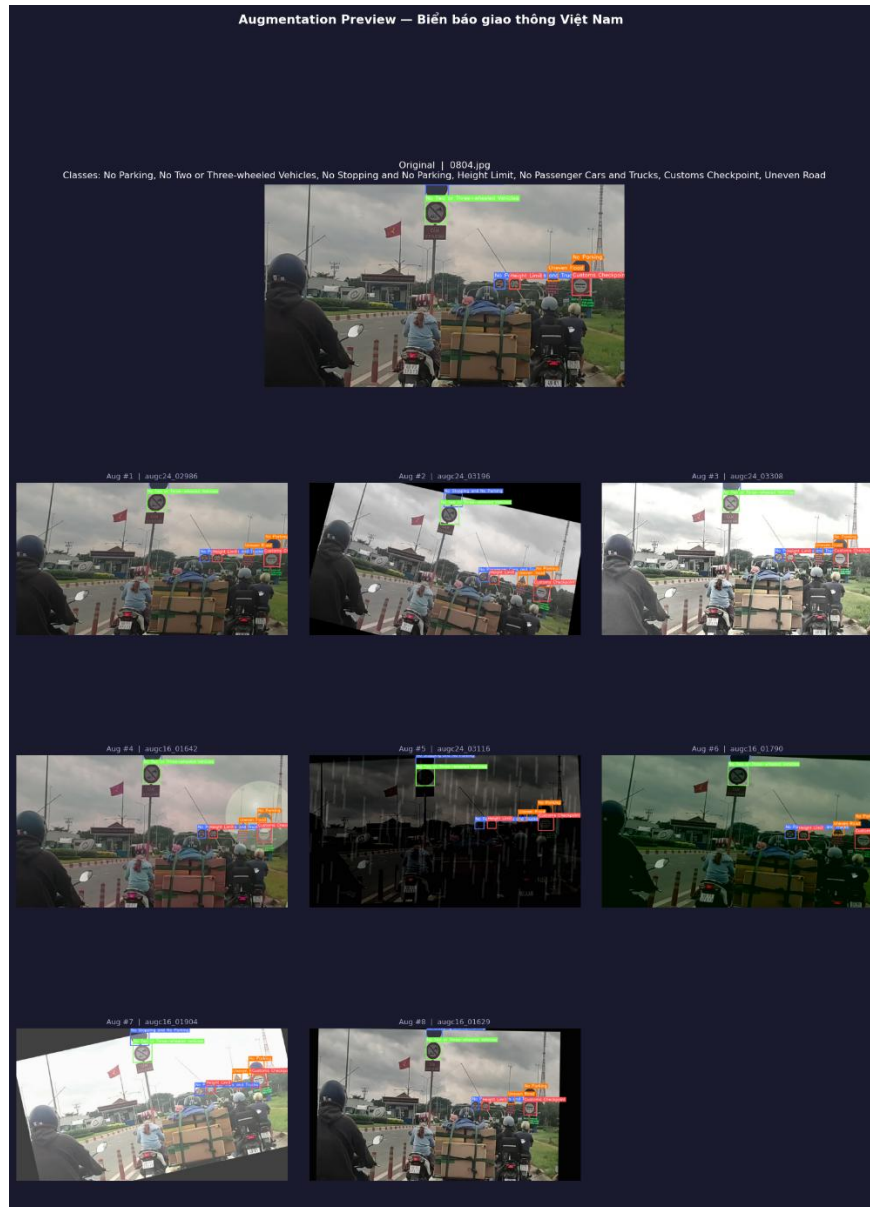
4.1.2. Phân Tích Tập Dữ Liệu Biển Báo Giao Thông Sau Augmentation

Việc áp dụng pipeline đã mang lại sự cải thiện vượt bậc về mặt định lượng, tạo ra một tập dữ liệu đủ mạnh để huấn luyện YOLOv8. Ngoài ra cũng cải thiện độ bao phủ thông qua Stratified Split, bắt buộc mỗi class phải có ít nhất 10 mẫu trong tập Valid và Test trước khi phân bổ phần còn lại cho tập Train.

Kết quả tăng cường các Class thiếu số đại diện:

- Class 44 (Other Danger): Từ 1 ảnh tăng lên 108 ảnh (Tăng 108 lần).
- Class 21 (Narrow Road on the Left): Từ 8 ảnh tăng lên 108 ảnh (Tăng 13,5 lần).
- Class 10 (No Stopping and No Parking): Giữ nguyên không augment do đã vượt ngưỡng 80 ảnh, tăng lên 2.271 ảnh chủ yếu do quá trình tái cấu trúc dataset. Ngoài ra

do có ảnh có những instances của các class thiếu hụt ảnh nên số lượng ảnh cũng tăng lên.



Hình 4.1. Ảnh gốc và các ảnh Augmentation trong tập dữ liệu thực tế

Sau khi áp dụng pipeline augmentation, mức độ mất cân bằng dữ liệu được cải thiện đáng kể. Tỉ lệ giữa class có nhiều ảnh nhất và ít ảnh nhất trong tập train giảm từ 825:1 (class 10 so với class 44) xuống còn khoảng 2271:108, tương đương tỉ lệ $\sim 21:1$. Mặc dù sự

mất cân bằng vẫn còn tồn tại, nhưng đã giảm từ mức cực đoan xuống mức có thể chấp nhận được với các kỹ thuật mô hình hóa hiện đại.

Quan trọng hơn, tất cả 67 class đều đạt số lượng ảnh đủ để huấn luyện và đánh giá: mỗi class có ít nhất 108 ảnh trong tập train, ít nhất 13 ảnh trong valid và ít nhất 13 ảnh trong test (theo thống kê thực tế). Điều này đảm bảo mô hình YOLOv8 có đủ mẫu để học đặc trưng của tất cả các biển báo, và quá trình đánh giá phản ánh đầy đủ khả năng nhận diện trên toàn bộ 67 class.

4.2. ĐÁNH GIÁ KẾT QUẢ HUẤN LUYỆN VÀ KIỂM TRA

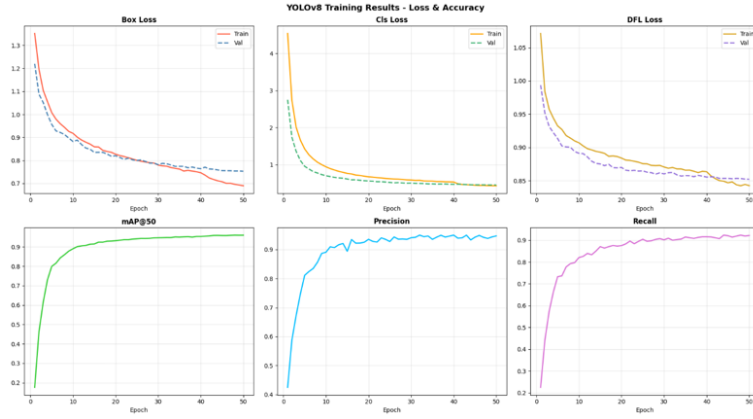
4.2.1. Phân tích độ chính xác và sai số huấn luyện

Quá trình thực nghiệm được tiến hành trên các biến thể kiến trúc YOLOv8 bao gồm: YOLOv8n (Nano), YOLOv8s (Small), YOLOv8m (Medium) và YOLOv8l (Large). Mục tiêu của chương này là phân tích sâu về hiệu năng học tập, độ chính xác chi tiết trên từng nhóm biển báo và so sánh đặc điểm kỹ thuật giữa các mô hình để tìm ra giải pháp tối ưu cho bài toán thực tế.

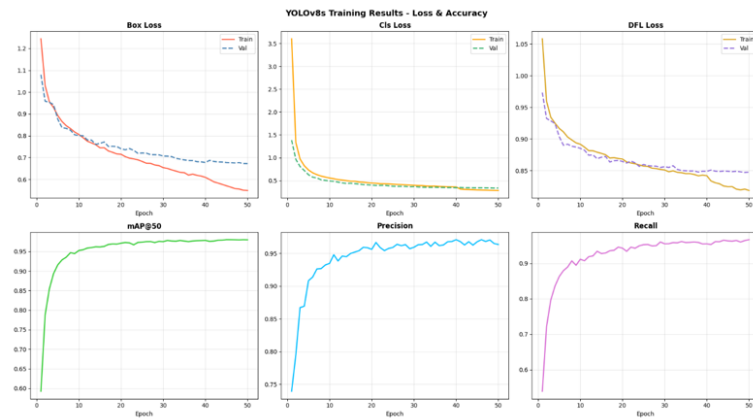
Dựa trên diễn biến từ đồ thị Training/Validation Loss và Accuracy, có thể đưa ra các phân tích kỹ thuật sau:

- **Sự hội tụ của hàm Loss:** Các thành phần sai số bao gồm Box loss, Class loss và DFL loss đều thể hiện tốc độ hội tụ nhanh trong 1/3 giai đoạn đầu của quá trình huấn luyện. Đặc biệt, sự sụt giảm mạnh của Box loss liên quan trực tiếp đến việc tối ưu hóa chỉ số IoU (Intersection over Union) giữa khung hình dự đoán và Ground Truth, giúp mô hình định vị chính xác tọa độ biển báo. Điều này đạt được nhờ chiến lược tăng cường dữ liệu đa dạng (mưa, sương mù, nhiều, che khuất), giúp mô hình học được các đặc trưng bền bỉ thay vì học vẹt dữ liệu gốc.

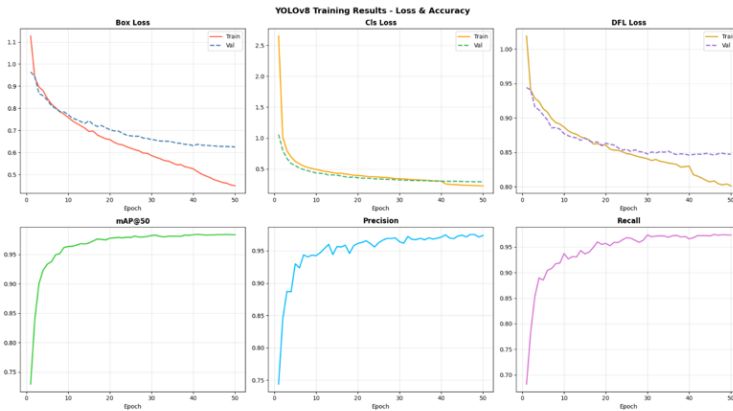
- **Tính ổn định và tổng quát hóa:** Cả hai chỉ số Training Loss và Validation Loss trên cả bốn mô hình đều cho thấy xu hướng giảm mạnh và đạt trạng thái hội tụ ổn định sau một khoảng thời gian huấn luyện nhất định. Việc đường Validation Loss bám sát và giảm đồng bộ với Training Loss là minh chứng cho thấy mô hình không gặp hiện tượng quá khớp (overfitting).
- **Hiệu suất mAP:**
 - Chỉ số $\text{mAP}@0.5$ phản ánh rõ nét sự cải thiện theo độ phức tạp của mô hình. YOLOv8n đạt $\text{mAP}@0.5 = 0.9247$ — mức nền tảng chấp nhận được với một mô hình nhỏ gọn, tuy nhiên vẫn còn khoảng cách đáng kể so với các biến thể lớn hơn. YOLOv8s cải thiện lên 0.9666, tương đương mức tăng ~ 4.5 điểm phần trăm, cho thấy việc tăng capacity mô hình mang lại hiệu quả rõ ràng ở ngưỡng $\text{IoU} = 0.5$. YOLOv8m đạt đỉnh cao nhất với $\text{mAP}@0.5 = 0.9731$, trong khi YOLOv8l đạt 0.9715 — thấp hơn YOLOv8m một lượng nhỏ (0.0016), cho thấy hiện tượng lợi nhuận giảm dần khi tiếp tục tăng quy mô mô hình trên tập dữ liệu này.
 - Đối với $\text{mAP}@0.5:0.95$ — chỉ số đánh giá nghiêm ngặt hơn khi tính trung bình qua nhiều ngưỡng IoU — xu hướng tương tự được ghi nhận: YOLOv8n đạt 0.7271, YOLOv8s đạt 0.7861, và hai mô hình lớn nhất đạt lần lượt 0.8132 (YOLOv8m) và 0.8165 (YOLOv8l). Đáng chú ý, ở chỉ số này YOLOv8l vượt nhẹ YOLOv8m (+0.0033), cho thấy mô hình lớn hơn có lợi thế trong việc định vị chính xác hơn ở các ngưỡng IoU cao. Precision và Recall của cả bốn mô hình đều duy trì ở mức cao (≥ 0.92), trong đó YOLOv8l đạt Recall cao nhất là 0.9771, phản ánh khả năng phát hiện toàn diện các biến báo với tỷ lệ bỏ sót thấp nhất.



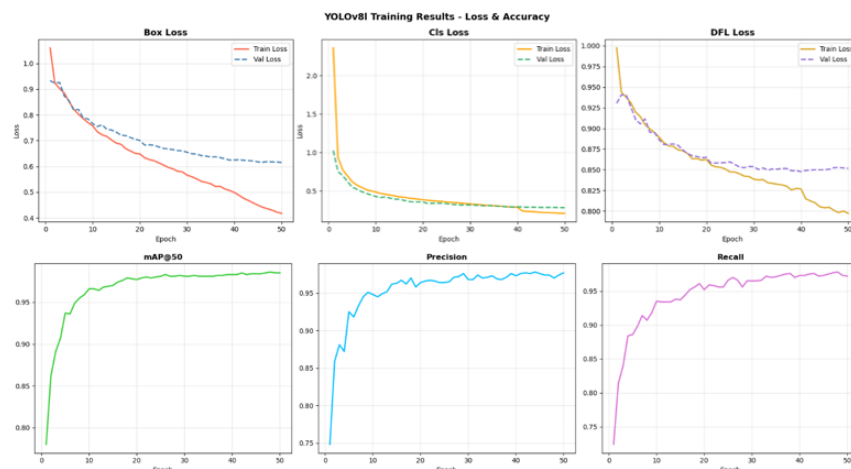
Hình 4.2. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8n



Hình 4.3. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8s



Hình 4.4. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8m



Hình 4.5. Kết quả đồ thị sai số và độ chính xác trong huấn luyện mô hình YOLOv8l

4.2.2. Phân Tích Hiệu Suất Chi Tiết Từng Class Qua Bar Chart

Để đánh giá chi tiết hơn hiệu suất của từng mô hình trên từng lớp biển báo, các biểu đồ Bar Chart per-class được xây dựng với hai panel song song. Panel trái thể hiện Precision (thanh xanh) và Recall (cam) của từng class, được sắp xếp theo thứ tự AP tăng dần từ trên xuống dưới, kết hợp đường kẻ dọc tham chiếu tại ngưỡng 90% giúp dễ dàng nhận diện các class chưa đạt yêu cầu. Panel phải trực quan hóa chỉ số AP@0.5 của từng class bằng hệ thống màu sắc ba cấp: xanh lá ($AP \geq 90\%$), vàng ($70\% \leq AP < 90\%$) và đỏ ($AP < 70\%$), với giá trị AP được ghi trực tiếp lên đầu mỗi thanh. Cách trình bày này giúp xác định nhanh các class mạnh, class trung bình và class yếu cần cải thiện trên toàn bộ 67 lớp biển báo giao thông Việt Nam.

Quan sát tổng thể trên cả bốn biểu đồ, có thể nhận thấy xu hướng rõ rệt: khi độ phức tạp mô hình tăng từ YOLOv8n lên YOLOv8m, phần màu xanh lá trong panel phải lan rộng dần, trong khi các thanh màu đỏ và vàng thu hẹp lại, phản ánh sự cải thiện đồng đều trên hầu hết các class.

Bảng dưới đây tổng hợp số lớp đạt ngưỡng $AP@0.5 \geq 95\%$ trên toàn bộ 67 class của từng mô hình:

Bảng 4.1. Thống kê các lớp đạt $AP@0.5 \geq 95\%$

Mô hình	Tổng số lớp	Lớp đạt $AP \geq 95\%$	Tỉ lệ
YOLOv8n	67	37	55.2%
YOLOv8s	67	54	80.6%
YOLOv8m	67	59	88.1%
YOLOv8l	67	57	85.1%

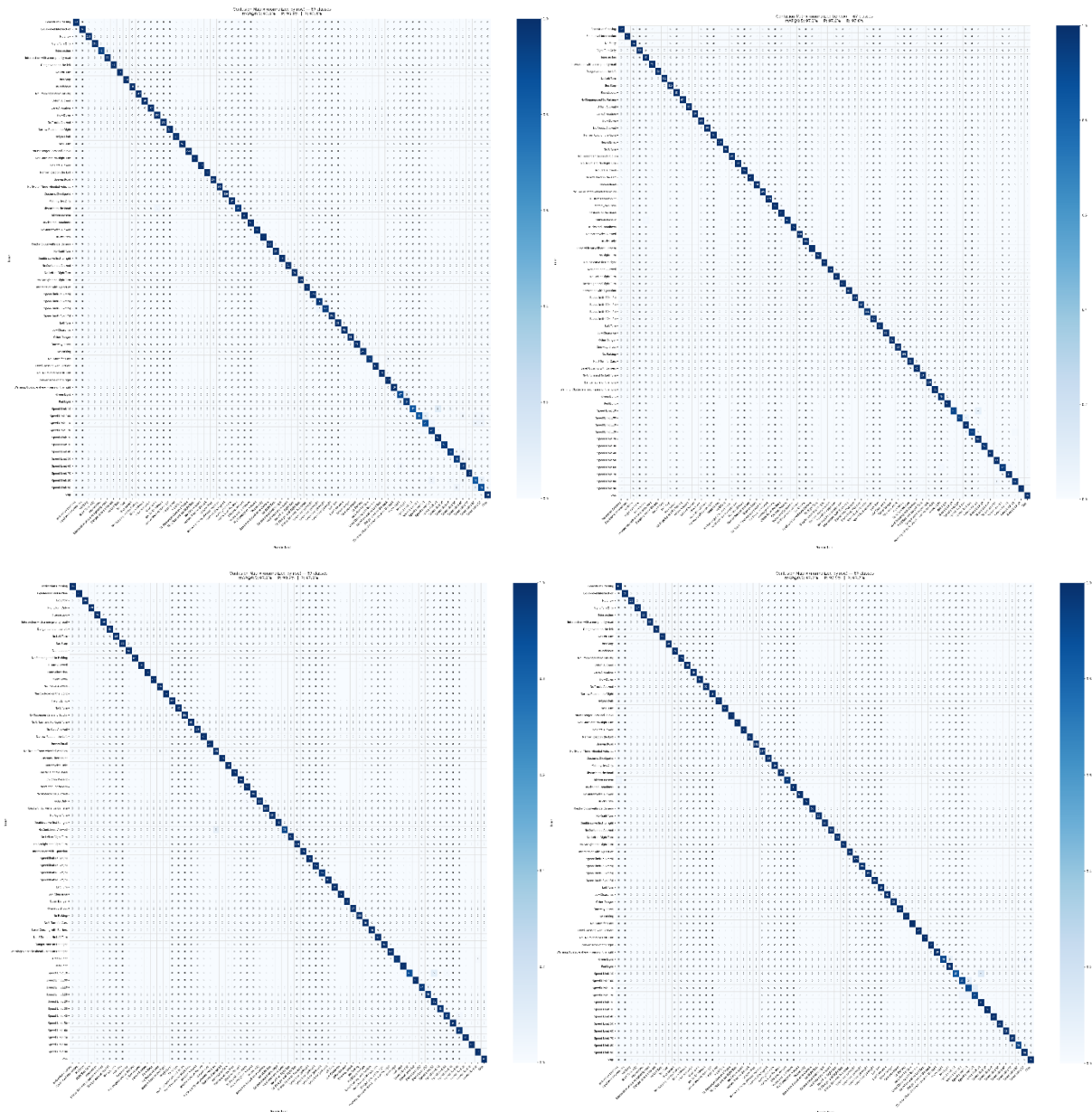
Kết quả cho thấy sự phân hóa rõ rệt giữa các mô hình. YOLOv8n chỉ đạt ngưỡng $AP \geq 95\%$ trên 37/67 class (55.2%), cho thấy với kiến trúc nhỏ gọn, mô hình này còn gặp khó khăn đáng kể trên gần một nửa số lớp biến báo, đặc biệt là các class có số lượng mẫu ít hoặc hình dạng tương đồng nhau. YOLOv8s cải thiện vượt bậc lên 54 class (80.6%), tương đương mức tăng 25.4 điểm phần trăm — mức cải thiện lớn nhất giữa hai bậc mô hình liền kề, cho thấy bước nhảy từ n lên s mang lại hiệu quả rõ ràng nhất.

YOLOv8m đạt kết quả tốt nhất với 59/67 class đạt $AP \geq 95\%$ (88.1%), khẳng định đây là điểm cân bằng tối ưu giữa độ chính xác và tài nguyên tính toán trong bộ bốn mô hình. YOLOv8l đạt 57 class (85.1%) — thấp hơn YOLOv8m 2 class — tiếp tục xác nhận hiện tượng lợi nhuận giảm dần đã quan sát được ở phần phân tích mAP tổng thể: việc tăng thêm tham số không đảm bảo cải thiện hiệu suất trên tất cả các class, và có thể do mô hình quá lớn so với quy mô tập dữ liệu hiện tại dẫn đến một số class bị suy giảm nhẹ.

Tuy nhiên, cần nhìn nhận YOLOv8l ở góc độ toàn diện hơn. Mặc dù số lớp đạt $AP \geq 95\%$ thấp hơn YOLOv8m, YOLOv8l là mô hình duy nhất trong bốn mô hình không có class nào rơi vào vùng đỏ ($AP < 70\%$), nghĩa là toàn bộ 67 class đều được nhận diện ở mức chấp nhận được trở lên. Điều này phản ánh tính ổn định và toàn diện của YOLOv8l — thay vì tập trung xuất sắc ở một số class và bỏ sót các class khó, mô hình lớn hơn có xu hướng phân bổ năng lực nhận diện đồng đều hơn trên toàn bộ phổ lớp. Đây là đặc tính quan trọng trong bài toán nhận diện biến báo giao thông thực tế, nơi việc bỏ sót hoàn toàn một loại biến báo có thể gây hậu quả nghiêm trọng hơn nhiều so với việc giảm nhẹ độ chính xác trên một số class nhất định.

4.2.3. Đánh Giá Ma Trận Nhầm Lẫn Của Dữ Liệu Kiểm Tra

Ma trận nhầm lẫn được chuẩn hóa theo hàng (normalized by row) trên toàn bộ 67 lớp, cho phép đánh giá trực quan tỉ lệ dự đoán đúng và các dạng nhầm lẫn phổ biến của từng mô hình.



Hình 4.6. Ma trận nhầm lẫn của bốn mô hình YOLO (lần lượt từ trái qua phải, từ trên xuống dưới là YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l)

Quan sát tổng thể, đường chéo chính của cả bốn ma trận đều thể hiện màu xanh đậm chiếm ưu thế, phản ánh tỉ lệ phân loại đúng cao trên hầu hết các class. Phần lớn các ô ngoài đường chéo có giá trị bằng 0 hoặc rất nhỏ, cho thấy các mô hình YOLOv8 được huấn luyện trên tập dữ liệu này có khả năng phân biệt tốt giữa các loại biển báo. Khu vực nhầm lẫn tập trung chủ yếu ở góc dưới bên phải của ma trận – đây là vùng chứa các class biển báo tốc độ (Speed Limit 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120), vốn có hình dạng rất tương đồng nhau, chỉ khác nhau ở con số bên trong.

Nhóm biển báo tốc độ là thách thức chung của cả 4 mô hình do đặc trưng hình ảnh tương đồng – cùng hình tròn viền đỏ, chỉ khác con số. Nhầm lẫn phổ biến nhất xảy ra giữa các cặp có giá trị tốc độ gần nhau như Speed Limit 110 ↔ Speed Limit 120, hoặc Speed Limit 10 ↔ Speed Limit 20.

Bảng 4.2. Kết quả đánh giá trên tập test

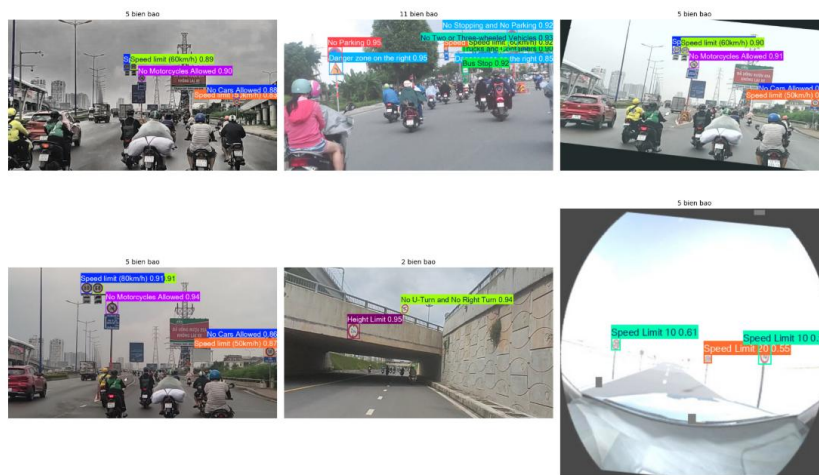
Mô hình	mAP@0.5	mAP@0.5:0.95	Precision	Recall
YOLOv8n	0.9247	0.7271	0.9533	0.9216
YOLOv8s	0.9666	0.7861	0.9693	0.9698
YOLOv8l	0.9731	0.8132	0.9742	0.9745
YOLOv8m	0.9715	0.8165	0.9754	0.9771

4.2.4. Kết Quả Nhận Diện Trên Tập Test

Kết quả cho thấy mô hình có khả năng phát hiện đồng thời nhiều biển báo trong cùng một khung hình – điển hình là ảnh giữa hàng trên với 11 biển báo được nhận diện cùng lúc, bao gồm các loại đa dạng như No Stopping and No Parking (0.92), No Parking (0.95), Danger zone on the right (0.95), Bus Stop (0.92), Trucks and Containers (0.90). Confidence score của phần lớn các dự đoán dao động trong khoảng 0.85–0.95, phản ánh độ tin cậy cao trong các điều kiện chụp ảnh đa dạng của dataset.

Trên các ảnh chụp đường đô thị và cao tốc, mô hình nhận diện ổn định các biển báo tốc độ (Speed limit 60km/h, 80km/h, 50km/h) và biển cấm (No Motorcycles Allowed, No

Cars Allowed) dù biển báo bị che khuất một phần hoặc nằm ở góc xa trong khung hình. Đáng chú ý, với ảnh chụp từ camera góc rộng ở góc dưới bên phải – vốn có méo hình học đáng kể – mô hình vẫn phát hiện được hai biển Speed Limit 10 (confidence 0.61 và 0.95) cùng biển Speed Limit 20 (0.55). Confidence thấp hơn ở ảnh này là hoàn toàn hợp lý do đặc điểm quang học của ống kính, tuy nhiên việc mô hình vẫn phát hiện đúng class cho thấy tính tổng quát hóa tốt trên các dạng ảnh đa dạng trong tập dữ liệu thực tế Việt Nam.



Hình 4.7. Kết quả nhận diện

4.3. ĐÁNH GIÁ TRADE-OFF

Việc lựa chọn mô hình YOLOv8 phù hợp với nhu cầu sử dụng không chỉ dựa trên độ chính xác mà cần suy xét đến những yếu tố khác (tốc độ suy luận, thời gian huấn luyện, kích thước mô hình và khả năng triển khai thực tế). Ở mục này, chúng em sẽ phân tích chi tiết những đánh đổi (trade-off) giữa bốn mô hình (YOLOv8n, YOLOv8m, YOLOv8s và YOLOv8l). Những mô hình này đã được huấn luyện trên cùng tập dữ liệu gồm 12.305 ảnh với 67 classes nhận diện biển báo giao thông, trong 50 epoch và sử dụng GPU Tesla T4.

4.3.1. Trade-off Giữa Accuracy và Speed

Kết quả đạt được sau khi huấn luyện cho thấy sự tỉ lệ nghịch giữa độ chính xác và tốc độ xử lý qua bốn mô hình.

Cụ thể, YOLOv8n đạt tốc độ inference nhanh nhất với 2.7ms cho mỗi ảnh (tương đương khoảng 370 FPS), đổi lại mAP@50-95 trên tập test chỉ đạt 74.7% – thấp nhất trong bốn mô hình. Mô hình YOLOv8s cải thiện được đáng kể độ chính xác (mAP@50-95 79.4%) và tốc độ inference đạt 6.6ms mỗi ảnh (cao hơn gấp khoảng 2.4 lần so với YOLOv8n). Ở YOLOv8m chỉ số mAP@50-95 tiếp tục nâng lên đến 81.9% nhưng cần đến 16.4ms cho mỗi ảnh (tốc độ chậm hơn 6 lần so với YOLOv8n). Đặc biệt, chỉ số mAP@50-95 của mô hình YOLOv8l chỉ cao hơn YOLOv8m một chút ở mức 82.3%, nhưng thời gian inference tăng đến 30.0ms (gấp hơn 11 lần so với YOLOv8n).

Với chỉ số mAP@50, chỉ số giữa các mô hình không có sự chênh lệch nhiều. YOLOv8n đạt 95.9%, YOLOv8s đạt 97.9%, YOLOv8m và YOLOv8l đều đạt 98.2%. Điều này cho thấy mô hình nhỏ nhất đã đạt ngưỡng nhận diện rất tốt, sự khác biệt chủ yếu thể hiện ở khả năng xác định bounding box (phản ánh qua mAP@50-95). Do đó, trong các ứng dụng yêu cầu đáp ứng thời gian thực (camera giao thông hoặc hệ thống ADAS), YOLOv8n hoặc YOLOv8s là lựa chọn hợp lý hơn so với YOLOv8l.

4.3.2. Trade-off Giữa Accuracy và Model Size

Kích thước của mô hình – đo bằng số lượng tham số và khối lượng tính toán (GFLOPs) – tăng theo bậc thang rõ rệt qua các mô hình. YOLOv8n là mô hình nhỏ nhất với 3 042 041 tham số và 8.3 GFLOPs. YOLOv8s lớn hơn xấp xỉ 3.7 lần về tham số (11 151 513) và gấp 3.4 lần về GFLOPs (28.6). Mô hình YOLOv8m có 25 878 553 tham số và 78.9 GFLOPs, lần lượt gấp 8.5 lần và 9.5 lần so với YOLOv8n. Lớn nhất trong 4 mô hình là YOLOv8l với 43 658 265 tham số và 165.1 GFLOPs – gấp hơn 14 lần về tham số và xấp xỉ 20 lần về GFLOPs so với mô hình YOLOv8n.

Mức tăng về độ chính xác không tỷ lệ thuận với mức tăng về tham số. Từ YOLOv8n lên YOLOv8s, số tham số tăng 3.7 lần nhưng mAP@50-95 chỉ tăng 4.7% (từ 74.7% lên 79.4%). Từ YOLOv8s lên YOLOv8m, số lượng tham số tăng thêm hơn 2 lần nhưng mAP chỉ tăng 2.5% (từ 79.4% lên 81.9%). Đặc biệt, từ YOLOv8m lên YOLOv8l, dù tham số tăng thêm 17.8 triệu (gấp 1.7 lần), nhưng mAP@50-95 chỉ tăng 0.4% (từ 81.9% lên 82.3%). Điều này cho thấy giới hạn lợi nhuận giảm dần (diminishing returns) khi tăng kích thước mô hình trong bài toán nhận diện biển báo giao thông. Trên thực tế, dung lượng file .pt của các mô hình là 6.2MB (YOLOv8n), 21.5MB (YOLOv8s), 49.7MB (YOLOv8m) và 83,7MB (YOLOv8l), tương ứng với kích thước trọng số tải về trong quá trình huấn luyện.

Bảng 4.3. Thống kê các chỉ số so sánh giữa bốn mô hình

Tiêu chí	YOLOv8n	YOLOv8s	YOLOv8m	YOLOv8l	Tốt nhất
Số tham số	3.042.041	11.151.513	25.878.553	43.658.265	YOLOv8n
GFLOPs	8.3	28.6	78.9	165.1	YOLOv8n
Thời gian train (50 epochs)	1.554 giờ	2.467 giờ	4.952 giờ	7.686 giờ	YOLOv8n
Tốc độ inference	2.7ms	6.6ms	16.4ms	30.0ms	YOLOv8n
mAP@50 (Test)	95.9%	97.9%	98.2%	98.2%	YOLOv8m/l
mAP@50-95 (Test)	74.7%	79.4%	81.9%	82.3%	YOLOv8l
Precision (Test)	95.1%	96.8%	97.7%	97.6%	YOLOv8m
Recall (Test)	91.6%	96.3%	96.8%	97.1%	YOLOv8l

4.3.3. Trade-off Giữa Triển Khai Thực Tế

Khi triển khai trên server hoặc hệ thống có GPU mạnh, YOLOv8m hoặc YOLOv8l là lựa chọn phù hợp nhất. Cả hai mô hình đều đạt $mAP@50$ 98.2% và $mAP@50-95$ lần lượt là 81.9% và 82.3%, cùng Precision và Recall đều trên 96%. Tuy thời gian inference lần lượt là 16.4ms và 30.0ms trên GPU Tesla T4, nhưng trong môi trường server hiện đại với GPU cao cấp hơn (như A100 hoặc RTX 4090), con số này sẽ giảm đáng kể. Hơn nữa, dung lượng lưu trữ hàng chục đến gần một trăm MB không phải là rào cản khi server có ổ đĩa dung lượng lớn. Trong trường hợp cần tối ưu thêm mà không muốn mất quá nhiều độ chính xác, YOLOv8m là lựa chọn cân bằng hơn so với YOLOv8l – đặc biệt khi mức tăng từ m lên l chỉ là 0.4% $mAP@50-95$ nhưng thời gian inference lại tăng gần gấp đôi.

Đối với triển khai trên thiết bị nhúng như Raspberry Pi, Jetson Nano, camera AI edge, hoặc các thiết bị di động có tài nguyên hạn chế, YOLOv8n nên là lựa chọn ưu tiên. Với chỉ 3 triệu tham số, 8.3 GFLOPs và tốc độ inference 2.7ms trên GPU, mô hình này đủ nhỏ gọn để chạy ổn định trên phần cứng yếu mà vẫn đạt $mAP@50 = 95,9\%$ – một con số rất cao cho bài toán nhận diện biển báo. File trọng số chỉ khoảng 6.2MB cũng giúp tiết kiệm bộ nhớ lưu trữ đáng kể. Nếu thiết bị nhúng có khả năng xử lý tốt hơn một chút (ví dụ Jetson Xavier hoặc các NPU thế hệ mới), YOLOv8s có thể là lựa chọn thay thế hợp lý, với sự cải thiện rõ ràng về Precision (từ 93.3% lên 96,8%) và Recall (từ 90.4% lên 96.3%) so với YOLOv8n trong điều kiện dữ liệu đa dạng hơn.

Xét tổng thể, trong bối cảnh bài toán nhận diện biển báo giao thông Việt Nam với 67 classes và yêu cầu ứng dụng thực tế, YOLOv8m là mô hình đáp ứng tốt nhất sự cân bằng giữa các yếu tố: đạt $mAP@50-95 = 81.9\%$ và $mAP@50 = 98.2\%$ với Precision 97.7% và Recall 96.8%, thời gian inference 16.4ms ở mức chấp nhận được cho nhiều nền tảng, số tham số ở ngưỡng trung bình (25.8 triệu) và thời gian huấn luyện khoảng 5 giờ – hợp lý cho quá trình phát triển và tinh chỉnh mô hình. Đây là lựa chọn được khuyến nghị cho hệ thống triển khai trên server hoặc các thiết bị có GPU tầm trung trở lên.

CHƯƠNG 5: KẾT LUẬN

Kết luận, đề tài “Phát hiện và phân loại biển báo giao thông Việt Nam sử dụng mô hình YOLOv8” đã đạt được các mục tiêu đề ra khi xây dựng thành công một hệ thống có khả năng nhận diện và phân loại biển báo với độ chính xác cao. Thông qua việc thu thập, tiền xử lý và tăng cường dữ liệu, nhóm đã tạo ra một bộ dữ liệu đa dạng và phù hợp với điều kiện thực tế tại Việt Nam. Quá trình huấn luyện và đánh giá trên nhiều phiên bản YOLOv8 (n, s, m, l) đã cho thấy sự khác biệt rõ ràng về hiệu năng, đồng thời làm nổi bật sự đánh đổi giữa tốc độ và độ chính xác.

Kết quả thực nghiệm chứng minh rằng các mô hình YOLOv8 có khả năng ứng dụng tốt trong bài toán nhận diện biển báo, đặc biệt trong các hệ thống giao thông thông minh và hỗ trợ lái xe. Tuy nhiên, đề tài vẫn còn một số hạn chế như quy mô dữ liệu chưa đủ lớn và chưa triển khai trên hệ thống thực tế.

Trong tương lai, hướng phát triển có thể tập trung vào việc mở rộng dataset, tối ưu mô hình cho thiết bị nhúng và triển khai thử nghiệm trong môi trường thực tế nhằm nâng cao tính ứng dụng của hệ thống.

TÀI LIỆU THAM KHẢO

[1] Link:

https://www.kaggle.com/datasets/pkdarabi/cardetection?fbclid=IwY2xjawRm4PlleHRuA2FlbQIxMABicmlkETFjVmVPRFB6M0pFWXo3YU45c3J0YwZhchBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHpWL5O-2IWpDX-yMiJ24HgWQm8VrrDCqRAvmFwMjzsru1SUc-u_UEoDnNSHu_aem_v8nOtN_88lJp9HeeXo4zuA

[2] Link:

https://docs.ultralytics.com/vi/models/yolov8/?fbclid=IwY2xjawRm4P1leHRuA2FlbQIxMABicmlkETFjVmVPRFB6M0pFWXo3YU45c3J0YwZhchBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHoCnXQ-6GoocyXV8SlF2QBxvDMm0s1Jsm84fiUzIkS_zfM_AIkYRGpN4BVZH_aem_RC5f2G_44VtwknBbLYb5Mw#supported-tasks-and-modes

[3] Link:

https://www.kaggle.com/datasets/maitam/vietnamese-traffic-signs?fbclid=IwY2xjawRm4QFleHRuA2FlbQIxMABicmlkETFjVmVPRFB6M0pFWXo3YU45c3J0YwZhchBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHnvDR04CePc7jZM4Cke7iT-az2o-oon25qCWQ__gtYNyQ9iwFv6p0tok8KIC_aem_QXPqBZPQuf9TE-n7PmZ-kA